PHANTOM OBSCURA

Complete Security Architecture Specification

Phantom Suite — Giblex Build Projects

Classification: Internal / Engineering Confidential

Version: 1.0 Draft | Date: March 2026

Phantom.Obscura

Zero-Knowledge Hardware-Bound Encrypted Vault Platform

Table of Contents

Document Information

Field	Value
Document Title	Phantom Obscura — Complete Security Architecture Specification
Project	Phantom Suite / Phantom Obscura
Build Path	O:\Users\Giblex\Build Projects\Phantom Suite\Phantom.Obscura
Version	1.0 Draft
Date	March 2026
Classification	Internal / Engineering Confidential
Author	Architecture synthesis from Kit.zip codebase + design sessions
Source Basis	Kit.zip (181 .cs source files), 10 PDF design documents, architectural conversations
Status	Living document — gaps noted in Section 25
Next Review	On completion of USB container enforcement milestone

Document Purpose

This document is the single authoritative specification for the Phantom Obscura security architecture. It consolidates and expands all prior design documents, whitepaper drafts, gap analyses, and architectural conversations into one comprehensive reference. Every design decision, cryptographic model, trust boundary, component interface, and engineering gap is described here at the level of detail required by engineers implementing the system.

Prior documents were fragmented, too short, or treated independently. This specification supersedes all of them. Where gaps exist between current code and target architecture, they are called out explicitly in Section 25 and addressed in the engineering roadmap in Section 26.

Scope

This specification covers:

- Full product definition and zero-knowledge posture
- All seven design principles with implementation implications
- All four trust domains and cross-boundary enforcement rules
- All eleven component architecture layers with full descriptions
- Code-aligned subsystem map from Kit.zip source files
- USB three-container topology with enforcement and co-location rules
- USB filesystem layout at path level

- Information placement matrix for every artifact class
- Vault manifest model and complete schema
- Per-object encryption model and key hierarchy
- Full cryptographic model including post-quantum hooks
- Signing and certificate trust model
- Thirteen-step unlock flow
- Runtime session architecture and sealed workspace model
- Plaintext containment requirements across all nine leak paths
- Runtime defense layer with code evidence
- Recovery architecture and deception layer
- Threat model and attack surface analysis
- Code-versus-target gap analysis
- Engineering roadmap and public positioning

Source Basis

All content is derived from: (1) the Kit.zip codebase comprising 181 C# source files representing the current Phantom Obscura implementation; (2) ten PDF architectural and design documents produced during design sessions; (3) the agreed architectural target established through detailed conversation-driven review. Where the code and the agreed target diverge, both states are documented and the gap is enumerated.

1 Product Definition

1.1 What Phantom Obscura Is

Phantom Obscura is a zero-knowledge, hardware-bound encrypted vault platform. It provides a secure environment in which sensitive objects — documents, credentials, secrets, and structured data — are stored, managed, and accessed exclusively under cryptographic control that is physically anchored to a dedicated USB device. The host machine running the application has no access to any plaintext at rest and derives no persistent cryptographic material from any operation.

The platform is designed around a single architectural premise: the only entity that can authorise decryption is the physical combination of the user's passphrase and the USB device. Remove either factor and the vault is permanently sealed to all parties, including Giblex, including the host operating system, and including any forensic examiner with full access to the host's storage.

1.2 Product Category

Phantom Obscura occupies a category that does not exist in commercially available software: a zero-knowledge, hardware-rooted personal vault with an encrypted manifest authority, per-object key isolation, and an active runtime defence layer. It is not a password manager, not a cloud storage service, not a general-purpose encryption wrapper, and not a hardware security module abstraction. It is closer in spirit to an offline HSM-backed vault than to any consumer product.

1.3 Zero-Knowledge Posture

Zero-knowledge, as used throughout this specification, means the following: at no point during normal vault operation does the host machine hold, write to disk, or transmit in any form the plaintext content of any vault object. The host may hold encrypted ciphertext transiently in application memory during a decryption operation, but that plaintext is wiped immediately after use and never written to any host-side storage medium, log, cache, preview store, temporary directory, or backup.

The zero-knowledge posture applies across all eleven layers of the component architecture and is enforced by the plaintext containment layer described in Section 18. Any component that writes decrypted content to the host without explicit policy authorisation is a violation of the zero-knowledge guarantee.

1.4 Hardware-Bound Trust Model

The trust anchor for every vault operation is the USB device. The USB device carries three encrypted containers, a hardware fingerprint record, and a hidden keyfile. None of these can be reconstructed from the host machine alone. This is not a software binding — the binding is

cryptographic: the Master Unlock Key is derived from material that physically lives on the USB device and cannot be reproduced without it.

Removing the USB device during a session immediately invalidates the session. The vault seals without user interaction. Objects in memory are wiped. The application returns to an unauthenticated state. This behaviour is enforced by the USB presence monitor, which is a continuous background service that cannot be disabled by user preference.

1.5 Manifest-Governed Model

All vault operations are governed by an encrypted manifest stored on the USB device. The manifest is not a simple directory listing — it is a cryptographic authority structure that contains the object registry, all wrapped per-object FileKeys, access policies, integrity hashes, Merkle references, signing references, rotation metadata, recovery references, and audit trail references. Nothing about the vault's contents is derivable from the host-side application state alone. Destroying the USB device destroys the manifest authority and renders all objects permanently inaccessible.

1.6 What Phantom Obscura Is Not

What it is NOT	Why this matters to the architecture
A cloud-synced vault	No network connectivity is required or permitted during vault operations. No plaintext or key material ever leaves the local device.
A password manager	Though credentials can be stored as vault objects, the platform is not optimised for credential management workflows. It has no browser integration.
A backup solution	The USB device is the vault. There is no secondary backup of the vault manifest or key material unless explicitly built into the recovery plane.
A general encryption utility	Obscura does not encrypt arbitrary files on the host filesystem. All encryption is scoped to vault objects within the three USB containers.
A shared team vault	The trust model is personal and hardware-bound. Multi-party access is not in scope and would require architectural changes to the key hierarchy.
Forensics-resistant hardware	Obscura relies on standard USB hardware. Physical attacks on the USB chip itself are outside the threat model. The software cryptographic model is the security boundary.
A compliance framework	Obscura does not implement FIPS 140-3 validated modules. The cryptographic primitives are algorithmically sound but not FIPS-validated.

2 Design Principles

Seven principles govern every architectural decision in Phantom Obscura. Each principle has direct implementation implications and is referenced throughout this specification wherever a design decision is explained.

Principle 1 — Zero-Knowledge by Architecture

The system is architected so that zero-knowledge is not a feature that can be disabled — it is a structural property. The host machine is architecturally incapable of retaining plaintext across session boundaries. This is achieved through: (a) all vault objects remaining encrypted on USB storage at all times; (b) decryption occurring only in application memory under session control; (c) decrypted buffers being explicitly zeroed immediately after use; and (d) the plaintext containment layer actively blocking all OS-level leak vectors.

Implementation implication: any component that writes to the host filesystem must write ciphertext only. Any component that holds decrypted data in memory must register that buffer with the session manager for mandatory zeroing on session close or USB removal.

Principle 2 — Hardware-Bound Trust

The USB device is not optional. It is not a convenience factor — it is the cryptographic trust anchor. The Master Unlock Key cannot be derived without material that physically resides on the USB device. This means that stealing the passphrase alone grants nothing. Stealing the USB device alone grants nothing. Only the combination, presented by the authorised user in a live session, unlocks the vault.

Implementation implication: the KDF inputs must include USB-resident material (USB secret, hardware fingerprint, hidden keyfile) that is never transmitted to or cached on the host. The USB presence monitor must be non-bypassable. The application must refuse to enter an authenticated session state if USB material cannot be verified.

Principle 3 — Manifest Authority

The vault manifest is the single source of truth for all vault state. It governs which objects exist, what keys decrypt them, what policies apply, and what the integrity baseline is. There is no host-side database, no registry entry, and no application-local index that can substitute for or augment the manifest. If the manifest says an object does not exist, the object does not exist from the application's perspective regardless of what physical data may be present in the object container.

Implementation implication: the manifest must be encrypted, integrity-protected, and signed. Loading a vault without a valid, signature-verified manifest must be refused. Any state that diverges between the in-memory session and the manifest on USB must be detected and treated as a tamper condition.

Principle 4 — Ephemeral Session Model

Decryption is always transient. An object is decrypted into application memory when it is needed, used, and then re-encrypted or discarded. There is no persistent decrypted form of any vault object. The session itself is bounded by USB presence: the moment the USB device is removed, the session is invalidated, all in-memory plaintext is zeroed, and the application returns to the unauthenticated state.

Implementation implication: the session manager must track all buffers containing decrypted material. Object lifecycle must include explicit open and close operations that trigger decryption and zeroing respectively. There must be no pathway by which a decrypted object outlives its containing session.

Principle 5 — Plaintext Containment

The operating system must not receive plaintext through any side channel. This includes: temporary file writes, autosave operations, clipboard history, search indexer captures, thumbnail generation, preview caching, pagefile/swap writes, crash dumps, and recent-files lists. Each of these is an active threat and must be explicitly mitigated at the application level. Passive reliance on the user not saving files is insufficient.

Implementation implication: the Runtime Defence layer must include active clipboard guards, file-write intercepts, memory locking (preventing page-out of sensitive buffers), indexing exclusions, and spawned-process tracking to prevent plaintext from escaping through child processes.

Principle 6 — Defence in Depth

No single security control is sufficient. Phantom Obscura layers cryptographic controls (encryption, signing, key wrapping), hardware controls (USB binding, fingerprint verification), application controls (session management, access policy), and operational controls (runtime defence, intrusion detection, deception) into an architecture where an attacker must defeat multiple independent layers to gain meaningful access.

Implementation implication: each layer must be independent. A failure in the clipboard guard must not weaken the encryption model. A failure in debugger detection must not weaken the key hierarchy. Layers must fail closed: on uncertainty, they must deny access and log the event.

Principle 7 — Graceful Degradation Under Partial Compromise

The system must degrade safely when components fail or are attacked. If the tamper detection system detects an anomaly, it must seal the vault and log the event — not crash with an unhandled exception that leaves plaintext in memory. If the USB device signals an unexpected read error, the session must close safely. If a wrapped key fails to unwrap, the affected object must be isolated, not the entire vault.

Implementation implication: every component must have a defined failure mode that results in vault sealing or object isolation, never in plaintext exposure. Error paths must be treated with the same security rigour as the happy path.

3 Trust Domains and Boundaries

Phantom Obscura defines four trust domains. Each domain has a distinct security posture, a distinct set of permitted operations, and a distinct set of trust-boundary crossing rules. Data and key material may only cross domain boundaries through explicitly defined, cryptographically enforced interfaces.

3.1 Domain Overview

Domain	Location	Trust Level	Primary Contents
Control Plane	Host machine	Low — hostile assumed	Application logic, UI, session state, policy enforcement engine
Trust Plane	USB device (root/vault containers)	High — hardware-anchored	Identity material, USB secret, hardware fingerprint, hidden keyfile, vault salt, signing keys, root certificate
Payload Plane	USB device (object container)	High — cryptographically bounded	Encrypted vault objects, manifest, wrapped FileKeys, integrity hashes
Recovery Plane	USB device (recovery container) + OOB	Constrained — audited access only	Recovery bundle, recovery codes (hash only), audit log, constrained session re-entry

3.2 Control Plane (Host Machine)

The Control Plane is the host machine. It is treated as hostile by the architecture: the host OS may be compromised, may have malware installed, may have a debugger attached, or may be running in a virtual machine. The application runs in the Control Plane but holds no persistent secrets and writes no plaintext to host storage.

Permitted in Control Plane: application code execution, UI rendering, session state management (in memory only), policy enforcement decisions, encrypted writes to USB, encrypted reads from USB, runtime defence monitoring.

Prohibited in Control Plane: plaintext object data at rest, passphrase storage beyond the KDF input phase, Master Unlock Key persistence, wrapped or unwrapped FileKey persistence beyond object access duration.

3.3 Trust Plane (USB — Root and Vault Containers)

The Trust Plane resides on the USB device in the root container and vault container. It contains all material from which the Master Unlock Key is derived. The Trust Plane is physically separate from the host; it cannot be accessed without USB device presence; and its contents are encrypted at rest with keys that are themselves derived from user passphrase plus device fingerprint.

Permitted in Trust Plane: USB identity record, hardware fingerprint, hidden keyfile, vault salt, Ed25519 signing keys (encrypted), root certificate, vault manifest (encrypted), policy metadata (encrypted).

Trust Plane material that crosses to the Control Plane: none in plaintext form. During unlock, specific KDF inputs are read from the Trust Plane and used immediately in the KDF operation. The outputs (MUK) exist in memory in the Control Plane for the duration of the session and are zeroed on session close.

3.4 Payload Plane (USB — Object Container)

The Payload Plane holds the encrypted vault objects, their metadata, and the per-object wrapped FileKeys. All material in the Payload Plane is encrypted. The Payload Plane is accessed through the Vault and Wrap layers, never directly from the application UI. No plaintext ever enters the Payload Plane; objects are encrypted before write and decrypted after read, with the decrypted form existing only in application memory.

3.5 Recovery Plane

The Recovery Plane is a constrained, audited trust domain that provides a time-limited and scope-limited pathway back into the vault when the standard unlock path is unavailable (e.g., passphrase forgotten, USB secret corrupted). The Recovery Plane operates under strictly different rules: every access is logged, recovery codes are single-use, and the scope of accessible objects during a recovery session may be restricted by policy. The Recovery Plane cannot bypass the hardware USB binding requirement.

3.6 Trust Boundary Crossing Rules

What crosses	Direction	Enforcement mechanism
KDF input material (USB secret, fingerprint, keyfile)	Trust Plane -> Control Plane (transient)	Read during unlock only; never cached; zeroed after MUK derivation
Master Unlock Key (MUK)	Derived in Control Plane	Memory-only; zeroed on session close or USB removal
Encrypted manifest bytes	Trust Plane -> Control Plane	Loaded for manifest decryption only; plaintext manifest held in memory; zeroed on session close
Per-object wrapped FileKey	Payload Plane -> Control Plane	Unwrapped using MUK; plaintext FileKey exists in memory only

		during object access window; zeroed immediately after
Decrypted object plaintext	Control Plane (memory only)	Never written to host storage; zeroed after use; session manager tracks all plaintext buffers
Encrypted ciphertext write	Control Plane -> Payload Plane	Re-encryption required before any write; no plaintext write path exists
Recovery bundle	Recovery Plane -> Control Plane	Decrypted only during audited recovery session; constrained object scope; mandatory audit log entry
Audit log entries	Control Plane -> Recovery Plane	Append-only; encrypted; integrity-protected; cannot be cleared without manifest re-signing

4 Full Component Architecture

The Phantom Obscura component architecture is organised into eleven layers. Each layer has a single responsibility, defined interfaces, and a specific security contract. Layers communicate through explicit interfaces — no layer may bypass a lower layer to access its material directly.

Layer 1 — Phantom Root (USB Identity and Certificate Anchor)

Phantom Root is the identity foundation of the system. It manages the USB device identity record, the hardware fingerprint binding, and the root certificate that anchors the trust chain for manifest and policy signing. Phantom Root is the first layer contacted during the unlock flow and the last verified during session close.

Responsibilities: USB device enumeration and selection, hardware fingerprint acquisition and verification, root certificate loading and validation, identity record encryption and decryption, certificate-chain verification for signing operations.

Security contract: if the hardware fingerprint does not match the stored record, unlock is refused and the event is logged. If the root certificate fails chain verification, all signing operations fail closed.

Layer 2 — Phantom Bind (USB Presence and Binding Enforcement)

Phantom Bind is the continuous USB presence enforcement layer. It is responsible for two things: (1) verifying during the unlock flow that the USB device matches the bound device profile; and (2) monitoring USB presence continuously during a live session, sealing the vault immediately on removal detection.

Responsibilities: device enumeration, binding profile match, real-time USB presence monitoring (polling or event-driven), session invalidation on removal, re-authentication gate on re-insertion.

Security contract: a session is only valid while the bound USB device is present. USB removal triggers immediate session seal with no grace period. Re-authentication from scratch is required if the USB device is re-inserted.

Layer 3 — Phantom Seal (Vault Sealing and MUK Derivation)

Phantom Seal owns the vault sealing lifecycle: taking the vault from sealed (USB present but MUK not derived) to unsealed (MUK derived, session active) and back to sealed. It orchestrates the Key Derivation Function inputs, derives the Master Unlock Key, and manages its in-memory lifecycle.

Responsibilities: passphrase collection and immediate KDF input (no storage), Argon2id orchestration, MUK derivation from all seven inputs, MUK memory management (SecureMemory allocation), session open/close triggering, MUK zeroing on seal.

Security contract: the passphrase is never stored beyond the moment it is passed to the KDF. The MUK is allocated in a SecureMemory region that prevents OS page-out and is explicitly zeroed on seal. The KDF parameters (memory, iterations, parallelism) must meet the configured minimum work factor.

Layer 4 — Phantom Vault (Manifest Authority and Object Registry)

Phantom Vault owns the vault manifest. It loads, decrypts, verifies, and maintains the in-memory manifest state. It is the gatekeeper for all object registry operations: adding, removing, or modifying objects requires a manifest update that is re-encrypted and re-signed before being written to USB.

Responsibilities: manifest decryption (AES-256-GCM using MUK), Ed25519 signature verification, in-memory manifest maintenance, object registry queries, policy metadata enforcement, manifest re-encryption and re-signing on any mutation, Merkle root verification.

Security contract: a manifest with a failing signature is never loaded. A manifest with a Merkle root mismatch is never trusted. Manifest writes are atomic: the old manifest remains valid until the new one is fully written and verified.

Layer 5 — Phantom Wrap (Key Wrapping and FileKey Management)

Phantom Wrap manages the per-object FileKey lifecycle. Each vault object is encrypted with a unique FileKey. The FileKey is wrapped (AES-256 key wrap or HKDF-derived and then encrypted) using the MUK and stored in the vault manifest. Phantom Wrap handles the wrap, unwrap, and rotation of FileKeys.

Responsibilities: FileKey generation (cryptographically random, per-object), FileKey wrapping using MUK, wrapped FileKey storage in manifest, FileKey unwrapping on object access, FileKey zeroing after object access, FileKey rotation on demand or on policy trigger.

Security contract: unwrapped FileKeys exist in SecureMemory only. A FileKey is zeroed as soon as the object access window closes. FileKey rotation requires manifest re-signing.

Layer 6 — Phantom Object (Object Lifecycle Management)

Phantom Object manages the complete lifecycle of vault objects: creation, read, update, and deletion. It coordinates with Phantom Wrap for key material and with Phantom Store for I/O. It enforces object-level access policies loaded from the manifest.

Responsibilities: object creation (generate ID, generate FileKey, encrypt, write to Payload Plane, update manifest), object read (decrypt into memory, enforce access policy), object update (re-encrypt, write, update manifest), object deletion (secure erase of Payload Plane data, manifest entry removal, FileKey zeroing), attachment and chunk management for large objects.

Layer 7 — Phantom Store (Encrypted Storage I/O)

Phantom Store provides the I/O interface between application layers and the USB containers. It ensures that all reads return ciphertext and all writes accept ciphertext. It has no knowledge of plaintext content — encryption and decryption happen in the layers above it.

Responsibilities: USB container file I/O, container integrity verification (HMAC or AEAD tag check at I/O boundary), write atomicity, container allocation management, chunked I/O for large objects.

Layer 8 — Phantom Session (Ephemeral Session Management)

Phantom Session owns the in-memory session state for an active vault unlock. It tracks all open objects, all plaintext buffers, all active timers, and all session-scope policies. It is the coordinator for session teardown: ensuring that every tracked buffer is zeroed, every open object is closed, and the MUK is returned to Phantom Seal for zeroing.

Responsibilities: session lifecycle (open, active, lock, close), buffer registry (tracking all SecureMemory allocations holding plaintext), idle timeout management, inactivity lock, object open/close tracking, session event audit logging.

Security contract: no plaintext buffer may be allocated outside the session buffer registry. On any session close path — normal, timeout, USB removal, tamper detection — all registered buffers are zeroed in order.

Layer 9 — Runtime Defence (Active Threat Detection and Response)

The Runtime Defence layer is a collection of active monitoring services that detect and respond to threats during a live session. It operates continuously and independently of the vault access operations. On detection of a threat condition, it triggers session seal before alerting the user.

Components: AdvancedDebuggerDetection.cs (debugger presence monitoring), VirtualMachineDetection.cs (VM environment detection), TamperDetectionService.cs (application code integrity monitoring), BuildIntegrityVerifier.cs (build hash verification at startup), ClipboardGuard.cs (clipboard write control), ClipboardHistoryExclusion.cs (Windows clipboard history suppression), ExportGuard.cs (file export policy enforcement), IntrusionService.cs (threat event aggregation and response), ThreatEvent.cs (threat event model), MemoryProtectionService.cs (process memory access control), SecureMemory.cs (page-locked memory allocation), WindowProtectionService.cs (window capture prevention), SpawnedProcessTracker.cs (child process monitoring and termination).

Layer 10 — Phantom Purge (Secure Deletion and Session Teardown)

Phantom Purge provides the secure deletion and zeroing operations used throughout the lifecycle. It is used by Phantom Session during teardown, by Phantom Object during object deletion, and by the Runtime Defence layer during threat response.

Responsibilities: multi-pass buffer zeroing (using platform-appropriate secure_zero_memory or equivalent), SecureMemory deallocation, file-level secure delete for USB-resident objects, session teardown sequencing.

Layer 11 — Phantom Trace (Audit Logging and Forensic Trail)

Phantom Trace provides the encrypted audit trail for all significant vault operations. Audit records are written to the USB device's encrypted audit log in the recovery container. The log is append-only from the application's perspective; clearing or truncating the log requires a manifest update with re-signing.

Responsibilities: structured audit event generation, event serialisation and encryption, append-only write to audit log on USB, integration with recovery audit service, session summary logging on close.

5 Code-Aligned Subsystem Map

The following table maps each architectural subsystem to its representative source files in Kit.zip. Files marked with (*) are present in the codebase but require modification to fully implement the target architecture. Files marked with (gap) are referenced by the architecture but not yet present in Kit.zip.

Subsystem	Layer	Representative .cs Files	Status
USB Identity	Phantom Root	UsbIdentityService.cs, HardwareFingerprint.cs	Present — needs container enforcement
Certificate Anchor	Phantom Root	RootCertificateService.cs, CertificateChainVerifier.cs	Present — needs binding to signing model
USB Binding	Phantom Bind	UsbBindingService.cs, DevicePresenceMonitor.cs	Present — continuous monitor needs strengthening
Device Removal	Phantom Bind	UsbRemovalHandler.cs, SessionInvalidator.cs	Partial — removal-to-seal latency not guaranteed
Vault Sealing	Phantom Seal	VaultSealService.cs, MasterUnlockKeyService.cs	Present — MUK lifecycle needs SecureMemory throughout
KDF Orchestration	Phantom Seal	KdfOrchestrator.cs, Argon2idService.cs, HkdfService.cs	Present — full 7-input model needs validation
Manifest Authority	Phantom Vault	VaultManifestService.cs, ManifestSerializer.cs	Present — physical separation to USB not enforced
Object Registry	Phantom Vault	ObjectRegistryService.cs, PolicyEnforcementService.cs	Present — Merkle verification gap
Manifest Integrity	Phantom Vault	ManifestIntegrityService.cs, MerkleService.cs	Partial — Merkle tree integration incomplete
Key Wrapping	Phantom Wrap	KeyWrappingService.cs, FileKeyService.cs	Present — per-object isolation

			needs verification
Key Rotation	Phantom Wrap	KeyRotationService.cs	Present — policy-triggered rotation not wired
Object Lifecycle	Phantom Object	ObjectEncryptionService.cs, VaultObjectService.cs	Present — chunking for large objects incomplete
Payload Addressing	Phantom Object	PayloadAddressingService.cs, ObjectMetadataService.cs	Present
Attachments	Phantom Object	AttachmentService.cs, ChunkingService.cs	Partial — chunk indexing not in manifest
Container I/O	Phantom Store	ContainerIoService.cs, UsbContainerManager.cs	Partial — three-container physical split not enforced
Storage Atomicity	Phantom Store	AtomicWriteService.cs	Gap — atomic write model not clearly implemented
Session Manager	Phantom Session	SessionManager.cs, EphemeralBufferService.cs	Present — buffer registry needs audit
Session Timers	Phantom Session	SessionTimerService.cs, IdleLockService.cs	Present
Debugger Detection	Runtime Defence	AdvancedDebuggerDetection.cs	Present — trigger-to-seal pathway needs wiring
VM Detection	Runtime Defence	VirtualMachineDetection.cs	Present — response action configurable
Tamper Detection	Runtime Defence	TamperDetectionService.cs	Present — build hash verification gap
Build Integrity	Runtime Defence	BuildIntegrityVerifier.cs	Present — sign-time hash not integrated

Clipboard Guard	Runtime Defence	ClipboardGuard.cs, ClipboardHistoryExclusion.cs	Present — Windows 11 history exclusion needs verification
Export Guard	Runtime Defence	ExportGuard.cs	Present — policy integration needed
Intrusion Service	Runtime Defence	IntrusionService.cs, ThreatEvent.cs	Present — event -> seal -> log pipeline needs end-to-end test
Memory Protection	Runtime Defence	MemoryProtectionService.cs, SecureMemory.cs	Present — page-lock coverage needs audit
Window Protection	Runtime Defence	WindowProtectionService.cs	Present
Process Tracking	Runtime Defence	SpawnedProcessTracker.cs	Present — child process plaintext pass-through gap
Secure Purge	Phantom Purge	SecurePurgeService.cs, ZeroingService.cs	Present — multi-pass overwrite not confirmed
Audit Logging	Phantom Trace	AuditLogService.cs, SessionAuditService.cs	Present — USB-side encrypted log write not wired
Recovery Session	Recovery Plane	RecoverySession.cs, ScopedRecoveryService.cs	Present — constrained scope not enforced by architecture
Recovery Audit	Recovery Plane	RecoveryAuditService.cs	Present — audit log linkage to main trace incomplete
Recovery Codes	Recovery Plane	RecoveryCodeService.cs	Present — single-use enforcement gap

Deception / Decoy	Deception Layer	DecoyVaultService.cs, DecoyCredentialGenerator.cs	Present — isolation from real vault needs enforcement
Passkey Extension	Extension Hooks	PasskeyService.cs	Present — not integrated into main unlock flow
YubiKey Extension	Extension Hooks	YubiKeyService.cs	Present — not integrated into main unlock flow
Post-Quantum Hooks	Crypto Layer	MlKemService.cs (referenced in README)	Gap — ML-KEM service not confirmed in Kit.zip

6 USB Deterministic Topology

The USB device carries three physically distinct encrypted containers plus a recovery container. Each container has a defined purpose, defined contents, and defined access rules. The three-container model is a structural security requirement, not an implementation convenience: it enforces separation between identity material, manifest authority, and payload data.

6.1 Container Definitions

Container	Purpose	Contents
Root Container	USB identity and KDF material anchor	USB identity record (encrypted), hardware fingerprint, hidden keyfile (entropy source), root certificate, root signing key (encrypted), vault KDF salt
Vault Container	Manifest authority	Encrypted vault manifest, manifest Ed25519 signature, policy metadata (encrypted), manifest version chain
Object Container	Vault payload storage	Encrypted vault object files, per-object encrypted metadata, per-object wrapped FileKey records, chunked payload segments
Recovery Container	Constrained re-entry and audit	Encrypted recovery bundle, recovery code hash table, encrypted audit log, session history records

6.2 Why Three Containers Are Necessary

Separating identity material from manifest authority from payload accomplishes several security goals simultaneously:

- Compartmentalisation: compromise of the object container (e.g., by extracting the USB and imaging the partition) reveals only ciphertext with no key material. The wrapped FileKeys in the manifest are inaccessible without the Root Container material.
- Manifest integrity: the vault manifest lives in its own container and its own file, signed separately. It cannot be replaced by rewriting the object container.
- Keyfile isolation: the hidden keyfile is in the Root Container, not in the manifest or object container. An attacker who obtains the manifest cannot derive the MUK without the keyfile.
- Attack surface reduction: a targeted attack on the manifest does not give access to the identity material needed to re-sign a forged manifest.

6.3 Container Co-Location Rules

All three primary containers (Root, Vault, Object) must reside on the same physical USB device. They must be present and pass integrity verification before the unlock flow advances beyond

the device binding phase. The application must refuse to open a vault if any container is missing or fails integrity verification.

The Recovery Container must also reside on the USB device. It is not separately accessible — it requires the same device to be present but uses a different access path than the standard unlock flow.

6.4 Container Enforcement Rules

Enforcement Rule	Mechanism
All three primary containers must be present on unlock	Container enumeration check in Phantom Bind before KDF begins
Root Container integrity must pass before KDF begins	AEAD tag verification on root container open; abort on failure
Vault Container signature must pass before session begins	Ed25519 signature verification on manifest; abort on failure
Object Container integrity verified per-object on access	Per-object AEAD tag check in Phantom Store; object access denied on failure
USB device fingerprint must match root container record	Fingerprint comparison in Phantom Root; abort and log on mismatch
No container may be written in plaintext at any point	Container I/O in Phantom Store enforces ciphertext-only writes
Container state must be consistent before session commit	Cross-container consistency check after manifest load; Merkle root comparison

6.5 Tiered Security and Storage Modes

Phantom Obscura supports three user-selectable security/storage modes. These modes do not change the cryptographic identity of the product; they change the visibility, storage transport, and operational assumptions of the USB trust anchor. The objective is to let users choose the right balance between secrecy, concealment, recoverability, cross-platform operability, and operational risk instead of treating every deployment as if it faces the same adversary.
The invariant across all three modes is the security core: the manifest remains the authority structure; the Master Unlock Key remains derived from passphrase plus USB-resident material; per-object FileKey isolation remains mandatory; Recovery remains a separate audited plane; and Attestor/Recovery remain sibling suite applications that integrate through the same vault identity and workspace contract. What changes by tier is the storage/transport layer: what the operating system can see, how much structure leaks before unlock, and what privileges or repair tooling are required to operate safely.

6.5.1 Current Code Alignment

The current codebase implements Tier 1 as the baseline operational model and already contains meaningful Tier 2 elements. The standard filesystem-backed layout remains authoritative, but the product now also supports hidden packed-volume layouts, sibling-suite launch and discovery for Phantom.Recovery and Phantom.Attestor, and a metadata-only credential index exported for Attestor inside the protected vault structure. In practical terms the live implementation is no longer a simple visible-folder vault: it is a filesystem-backed encrypted architecture with growing concealment and cross-app integration layers.
This distinction matters for the specification. Tier 1 is not a legacy or weak mode; it is the supportable, production-oriented foundation. Tier 2 builds on it by minimising visible structure and making packed or decoy-backed storage the default. Tier 3 is the expert hostile-environment mode in which the operating system is deliberately denied any meaningful filesystem interpretation of the vault medium.

6.5.2 Mode Invariants

Manifest authority remains mandatory in every tier. No tier allows a host-side database or alternate index to replace the encrypted manifest.
Key hierarchy remains constant in every tier: passphrase plus USB-resident factors derive the MUK; the MUK unwraps or derives per-object FileKeys; payload remains encrypted at rest at all times.
Recovery remains hardware-bound and separately audited in every tier. A higher concealment tier must not create an undocumented bypass around the Recovery Plane.
Attestor and Recovery are suite siblings, not optional abstractions. Every tier must preserve the ability to resolve the same vault identity, locate the same recovery workspace, and exchange only metadata or explicitly authorised recovery/bootstrap artefacts.
All tiers must preserve authenticated encryption, anti-rollback manifest handling, explicit seal-on-USB-removal behaviour, and plaintext containment on the host.

6.5.3 Tier 1 — Standard Secure

Tier 1 is cryptographic denial with normal removable-media behaviour. The USB mounts as a standard filesystem-backed device. The operating system may enumerate a hidden .phantom directory, packed master-volume file, encrypted manifests, encrypted object blobs, recovery artifacts, and trace artifacts. An attacker can infer that a security product is present and can learn rough file sizes, timestamps, and change cadence, but cannot derive the MUK, read

object plaintext, or forge the manifest without defeating both AEAD integrity and the signing model.
Privacy and security value: Tier 1 provides strong confidentiality, strong integrity, clean supportability, and the lowest corruption risk. It is the best mode for most users, the easiest mode for backup, migration, support, Recovery, and Attestor integration, and the safest mode for broad cross-platform use. Its primary weakness is not cryptographic weakness; it is structural visibility. A curious examiner can tell that a protected vault exists even if the contents are unreadable.
Mandatory controls in Tier 1: authenticated encryption for all artifacts; hidden/system attributes for the Phantom storage root; encrypted manifest plus detached signature; USB presence enforcement; device binding; secure backup/export; recovery workspace contract; and explicit host-side plaintext containment. Tier 1 should be the default provisioning choice and the recommended mode for users who need security without specialist storage risk.

6.5.4 Tier 2 — Stealth Secure

Tier 2 adds concealment to Tier 1 without yet abandoning the filesystem-backed operational model. In this mode the filesystem remains present for portability and support, but the meaningful Phantom structure is minimised or packed so that the operating system sees only a generic or decoy surface, a packed master volume, or an innocuous layout with far less product fingerprinting. The true manifest, object payloads, and suite metadata remain encrypted exactly as in Tier 1; the change is that artifact discovery becomes materially harder before unlock.
Privacy and security value: Tier 2 is the strongest balance mode. It materially reduces metadata leakage, reduces obvious product fingerprinting, and improves plausible deniability while preserving far more recoverability and cross-platform operability than raw-device mode. This is the natural home for packed master volumes, metadata-minimised credential indexes, optional decoy-visible surfaces, and stricter artifact placement rules. The main trade-off is complexity: concealment logic, packing/unpacking rules, decoy behavior, and suite discovery contracts must all remain internally consistent or the mode becomes brittle.
Mandatory controls in Tier 2: every Tier 1 control plus mandatory packed master volume support, visible metadata minimisation, constrained indexing/export artifacts, stricter artifact naming and placement obfuscation, and optional but policy-driven decoy surfaces. Tier 2 should be recommended for privacy-focused users, field operators, and users who care not only about secrecy of contents but also about reducing discoverability of the vault itself.

6.5.5 Tier 3 — Black Secure

Tier 3 replaces filesystem readability with raw-device or non-standard layout semantics. In this mode the operating system should not be able to interpret the device as a normal data volume and will typically present it as an unknown disk or prompt for formatting. Phantom Obscura, however, still resolves the same logical container set — root, vault, object, and recovery planes — by reading deterministic offsets or a raw-device map rather than ordinary paths. This is presentation-layer denial on top of cryptographic denial: the attacker is denied both plaintext and normal structural inspection.
Privacy and security value: Tier 3 provides the strongest concealment and the lowest pre-unlock metadata leakage. It is the best fit for adversarial environments where the mere existence of a secure vault is sensitive. It also carries the highest engineering and operational risk. Raw-device mode requires privileged I/O, transactional writes, journal or repair structures, careful anti-corruption design, expert-grade migration and backup tooling, and explicit user education. If implemented carelessly, Tier 3 can become less safe overall than Tier 1 because storage-engine failure risk starts to compete with attacker risk.

Mandatory controls in Tier 3: every Tier 2 control plus raw block layout management, authenticated header and footer maps, transactional or journaled writes, repair tooling, privileged device access, guarded backup/export flows, and expert-mode warnings during provisioning and migration. Tier 3 must be opt-in, clearly labelled as expert-only, and never silently selected by the application.

6.5.6 Comparative Security and Privacy Outcome

The three modes are not a simple ascending ladder of “good, better, best.” Tier 1 is strongest on reliability and supportability. Tier 2 is strongest on overall balance between secrecy, concealment, and survivability. Tier 3 is strongest on concealment and OS-level visibility denial. The right choice depends on the threat model: if the main risk is theft of data, Tier 1 may already be sufficient; if the risk includes scrutiny, targeting, or coercive inspection of the media, Tier 2 becomes materially stronger; if the risk includes hostile forensic discovery where even visible encrypted structure is unacceptable, Tier 3 becomes justified.

For product positioning, Obscura should present the modes as user-selectable security postures rather than implying that lower modes are “insecure.” Tier 1 protects secrets. Tier 2 protects secrets and conceals structure more effectively. Tier 3 protects secrets, conceals structure aggressively, and assumes the user accepts higher operational burden in exchange for adversarial-grade deniability and visibility resistance.

6.5.7 Comparative Decision Matrix

The following matrix summarises the operational meaning of each mode. It should be treated as the quick-selection view for provisioning, migration, support, and public product guidance.

Dimension	Tier 1 — Standard Secure	Tier 2 — Stealth Secure	Tier 3 — Black Secure
OS visibility	Normal removable-media mount; hidden or packed encrypted artifacts still enumerable	Filesystem-backed but meaningful structure minimised, packed, or concealed behind generic or decoy surface	OS should not interpret the medium as a normal readable volume
What an attacker learns pre-unlock	Product presence, rough file sizes, timestamps, and change cadence	Far less product fingerprinting; existence of meaningful vault structure is harder to confirm	Primarily device size and presence of a non-standard layout
Forensic resistance	Strong cryptographic secrecy, moderate structural privacy	Strong cryptographic secrecy plus materially stronger structural privacy	Strongest structural privacy and concealment
Corruption and implementation risk	Lowest	Moderate	Highest
Backup and recovery difficulty	Lowest friction; most supportable	Moderate; requires packed/decoy-aware tooling	Highest; requires specialised backup, journal, and repair tooling
Privilege requirements	Usually ordinary user-space file access	Usually ordinary user-space file access with some	Likely privileged raw-device access for provisioning,

		elevated packaging or device tasks	migration, and repair
Cross-app suite compatibility	Best baseline for Obscura, Recovery, and Attestor interoperability	Good, provided suite metadata and staging rules are strictly maintained	Must be explicitly designed; sibling apps cannot rely on filesystem semantics
Recommended user profile	Most users; production default	Privacy-focused and field users needing stronger concealment	Expert users in adversarial or inspection-heavy environments
Primary strength	Reliability and supportability	Best balance of secrecy, concealment, and survivability	Maximum concealment and visibility denial

6.5.8 Provisioning and Migration Rules

Mode selection should occur at provisioning time and be stored as part of the vault identity and policy metadata so every sibling app in Phantom Suite resolves the same storage expectations. Migration between modes is permitted but must be explicit, audited, and transactional. Up-tier migration (Tier 1 to Tier 2, Tier 2 to Tier 3) changes the storage transport while preserving manifest authority, key hierarchy, and object identity. Down-tier migration must warn the user that the resulting medium may become more visible to the host operating system or to forensic inspection.

Recovery and Attestor integration must remain mode-aware. Recovery resolution must continue to derive the same recovery workspace and vault record regardless of storage tier, and Attestor search/link flows must continue to consume only metadata explicitly staged for cross-app use. No tier may rely on sibling apps parsing plaintext vault structure opportunistically from the USB. Cross-app integration must always resolve through the suite contract, not by guessing filesystem state.

7 USB Filesystem Layout

The following is the canonical filesystem layout for the filesystem-backed Phantom Obscura modes (Tier 1 Standard Secure and Tier 2 Stealth Secure). In those modes all vault data resides under the hidden .phantom directory or its packed master-volume equivalent at the root of the USB volume. Tier 3 Black Secure replaces the visible filesystem contract with a raw-device layout as defined in Section 6.5. No plaintext data is stored anywhere on the USB device; all files are encrypted at rest.

Canonical Path Structure

```
/usb/   .phantom/     root/       identity.enc       keyfile.bin       fingerprint.bin
root.cert       root.key.enc       salt.bin     vault/       manifest.enc
manifest.sig       policy.enc       manifest.chain.enc     objects/       {object-
uuid}/         payload.enc         meta.enc         filekey.enc         chunks/
chunk_0001.enc           chunk_0002.enc           ...     recovery/       bundle.enc
codes.hash       audit.log.enc       session_history.enc     trace/
session.log.enc       event.log.enc
```

7.1 Path-Level Artifact Allocation

Path	Contents	Access Layer
/usb/.phantom/root/identity.enc	USB identity record — device ID, binding profile, creation timestamp (AES-256-GCM encrypted)	Phantom Root
/usb/.phantom/root/keyfile.bin	Hidden keyfile — 64 bytes of cryptographically random entropy, written once at vault creation	Phantom Root (read-only after creation)
/usb/.phantom/root/fingerprint.bin	USB hardware fingerprint — serialised hardware attributes, hashed for comparison	Phantom Root
/usb/.phantom/root/root.cert	Root X.509 certificate for the trust chain; used to verify signing key certificates	Phantom Root
/usb/.phantom/root/root.key.enc	Ed25519 root signing key (encrypted with MUK); used to sign manifests and policies	Phantom Root / Phantom Seal
/usb/.phantom/root/salt.bin	32-byte Argon2id KDF salt; unique per vault; written at creation; never changed except on explicit vault re-key	Phantom Seal
/usb/.phantom/vault/manifest.enc	Encrypted vault manifest (AES-256-GCM, key = MUK); contains full object registry, wrapped FileKeys, policy, integrity state	Phantom Vault
/usb/.phantom/vault/manifest.sig	Ed25519 detached signature over the encrypted manifest bytes; verified before manifest is loaded	Phantom Vault
/usb/.phantom/vault/policy.enc	Vault-level policy metadata: export policy, recovery policy, session	Phantom Vault

	timeout config, object access policies	
/usb/.phantom/vault/manifest.chain.enc	Historical manifest version chain for audit and rollback detection	Phantom Vault
/usb/.phantom/objects/{uuid}/payload.enc	Encrypted object payload (AES-256-GCM, key = FileKey); complete object content for small objects	Phantom Store
/usb/.phantom/objects/{uuid}/meta.enc	Encrypted object metadata: object name, type, tags, timestamps, content-type, size, chunk manifest	Phantom Object
/usb/.phantom/objects/{uuid}/filekey.enc	Per-object FileKey wrapped with MUK using AES-256-KW; separate file for clean key rotation	Phantom Wrap
/usb/.phantom/objects/{uuid}/chunks/chunk_NNNN.enc	Chunked payload segments for objects exceeding the single-file size threshold	Phantom Store
/usb/.phantom/recovery/bundle.enc	Encrypted recovery bundle: contains constrained recovery key material, scope limits, expiry	Recovery Plane
/usb/.phantom/recovery/codes.hash	PBKDF2-hashed recovery code verifiers; the actual codes are never stored; hashes only	Recovery Plane
/usb/.phantom/recovery/audit.log.enc	Encrypted append-only audit log for all recovery plane access events	Phantom Trace / Recovery Audit
/usb/.phantom/recovery/session_history.enc	Encrypted session history for recovery sessions: timestamp, scope, outcome	Recovery Plane
/usb/.phantom/trace/session.log.enc	Encrypted per-session audit entries: open/close timestamps, objects	Phantom Trace

	accessed, anomaly events	
/usb/.phantom/trace/event.log.enc	Encrypted event log for runtime defence events: threat events, intrusion events, tamper detections	Phantom Trace / Runtime Defence

8 Information Placement Matrix

The information placement matrix defines where every artifact class in the Phantom Obscura system lives at rest, what form it takes, and what access is required to read it. Nothing in this matrix should ever be in plaintext on the host machine.

Artifact Class	Location at Rest	Form	Access Requirement
User Passphrase	Never stored	Memory only (KDF input)	User input per session
USB Secret	USB Root Container	Encrypted (AES-256-GCM)	MUK derivation process
USB Hardware Fingerprint	USB Root Container	Encrypted hash	Physical USB presence
Hidden Keyfile	USB Root Container	Encrypted binary blob	Physical USB presence + MUK
Vault KDF Salt	USB Root Container	Plaintext binary (non-secret)	Physical USB presence
Master Unlock Key	Application memory only	SecureMemory allocation (page-locked)	Active unlock session
Vault Manifest (plaintext)	Application memory only	Deserialised struct in SecureMemory	Active unlock session
Vault Manifest (ciphertext)	USB Vault Container	AES-256-GCM encrypted	Physical USB presence
Manifest Signature	USB Vault Container	Ed25519 detached signature	Loaded with manifest
Per-Object FileKey (unwrapped)	Application memory only	SecureMemory — duration of object access	Active session + object open
Per-Object FileKey (wrapped)	USB Object Container (filekey.enc)	AES-256-KW wrapped	Active session + manifest
Object Payload (plaintext)	Application memory only	SecureMemory — duration of object access	Active session + object open + FileKey
Object Payload (ciphertext)	USB Object Container	AES-256-GCM encrypted	Physical USB presence
Object Metadata (plaintext)	Application memory only	Transient deserialised struct	Active session

Object Metadata (ciphertext)	USB Object Container (meta.enc)	AES-256-GCM encrypted	Physical USB presence
Root Certificate	USB Root Container	X.509 PEM/DER (non-secret)	Physical USB presence
Root Signing Key (plaintext)	Application memory only	SecureMemory — signing operations only	Active session + MUK
Root Signing Key (ciphertext)	USB Root Container	AES-256-GCM encrypted	Physical USB presence
Recovery Bundle	USB Recovery Container	AES-256-GCM encrypted	Recovery session + recovery code
Recovery Code (user-held)	Out of band	Never stored by application	User-held physical/printed record
Recovery Code Verifier	USB Recovery Container	PBKDF2 hash only	Recovery session
Audit Log	USB Recovery/Trace Containers	AES-256-GCM encrypted, append-only	Active session (append) / Recovery (read)
Session Event Log	USB Trace Container	AES-256-GCM encrypted	Active session (append) / Recovery (read)
Decoy Vault Credentials	Application memory (decoy session)	Synthetic, non-real credentials	Decoy trigger condition
Build Integrity Hash	Embedded in binary	SHA-256 of canonical build artefacts	Verified at startup by BuildIntegrityVerifier.cs

9 Vault Manifest Model

9.1 The Manifest as Encrypted Authority

The vault manifest is the cryptographic authority structure for the entire vault. It is not a directory listing, not a database index, and not an application configuration file. It is the single encrypted record that governs what objects exist, what keys decrypt them, what policies govern access, and what the integrity baseline is for the entire vault state.

The manifest is stored on the USB device in the Vault Container, encrypted with the Master Unlock Key using AES-256-GCM. Its integrity is protected by the AEAD authentication tag. Its authenticity is protected by an Ed25519 detached signature computed over the encrypted manifest bytes, stored separately in manifest.sig. This means: to forge a manifest, an attacker would need both the MUK (to produce a valid AEAD tag) and the Ed25519 private signing key (to produce a valid signature). Neither is available to an attacker who has only the USB device and no passphrase.

9.2 What the Manifest Governs

- Object registry: the canonical list of all vault objects, including their UUIDs, names, types, sizes, creation and modification timestamps, and references to their payload locations in the object container.
- Wrapped key records: for every object in the registry, the manifest contains the AES-256-KW wrapped FileKey. The FileKey is the only key that can decrypt the object payload. The wrapped FileKey cannot be unwrapped without the MUK.
- Policy metadata: vault-level and object-level access policies. These include export policy (whether an object can be exported to the host), session policy (whether an object requires re-authentication), and recovery scope policy (whether an object is accessible during a recovery session).
- Integrity hashes: a SHA-256 or BLAKE3 hash of each object's ciphertext. Used to verify object integrity on access and to detect tampering with the object container.
- Merkle references: a Merkle tree root computed over all object integrity hashes. Used to verify the consistency of the entire vault state with a single comparison.
- Signing references: references to the signing certificate and key version used to produce the manifest signature. Required for key rotation and certificate chain validation.
- Rotation metadata: history of FileKey rotations, MUK re-key events, and signing key rotations. Used to verify that a manifest is current and has not been replaced with an older version.
- Recovery references: references to the recovery bundle location and recovery code verifier table. Used by the recovery plane to locate and validate recovery material.
- Audit references: references to the audit log location and the last-committed audit record hash. Used to verify audit log continuity.

9.3 Manifest Lifecycle

The manifest is loaded exactly once per unlock session, during the unlock flow after MUK derivation. It is decrypted, signature-verified, Merkle-root-verified, and deserialised into application memory. It lives in a SecureMemory allocation for the duration of the session. Any mutation (object add, delete, update, key rotation) requires: (1) the in-memory manifest to be updated; (2) the updated manifest to be re-serialised; (3) the serialised bytes to be re-encrypted with the MUK; (4) the new ciphertext to be signed with the Ed25519 signing key; (5) the new ciphertext and signature to be written atomically to the Vault Container. The old manifest is not overwritten until the new write is confirmed.

On session close, the in-memory manifest is zeroed by Phantom Purge. The USB-side manifest remains as the authoritative record.

10 Full Vault Manifest Schema

The following represents the complete vault manifest schema. Field names use snake_case. All byte-array fields are base64-encoded in the serialised form. Timestamps use ISO 8601 UTC format.

10.1 Top-Level Manifest Structure

Field	Type / Size	Description
schema_version	string	Manifest schema version (e.g. '2.0'). Used for migration.
vault_id	UUID (16 bytes)	Unique vault identity. Generated at vault creation. Immutable.
vault_name	string (encrypted)	User-visible vault name. Stored in encrypted form.
created_at	ISO 8601 UTC	Vault creation timestamp.
last_modified	ISO 8601 UTC	Timestamp of last manifest write.
manifest_sequence	uint64	Monotonically increasing write counter. Used to detect rollback.
usb_binding_profile	object	USB device binding record (see 10.2)
kdf_parameters	object	Argon2id KDF parameters (see 10.3)
object_registry	array	Array of object registry entries (see 10.4)
wrapped_key_records	array	Array of wrapped FileKey records (see 10.5)
policy_metadata	object	Vault-level and object-level policy configuration (see 10.6)
integrity_hashes	map[uuid -> hash]	SHA-256 hash of each object's ciphertext. Keyed by object UUID.
merkle_root	bytes (32)	BLAKE3/SHA-256 Merkle root over all integrity hashes.
merkle_tree_ref	string	Path to full Merkle tree serialisation (if stored separately).
signing_ref	object	Signing key version and certificate reference (see 10.7)
rotation_metadata	object	Key rotation and re-key history (see 10.8)

recovery_refs	object	Recovery bundle and code verifier references (see 10.9)
audit_refs	object	Audit log location and last record hash (see 10.10)

10.2 USB Binding Profile

Field	Description
device_id	USB device serial number (hashed — do not store raw serial)
fingerprint_hash	SHA-256 of hardware fingerprint bytes captured at vault creation
fingerprint_algorithm	Algorithm used for fingerprint (e.g. 'SHA-256-CONCAT-ATTRS')
bound_at	Timestamp of initial binding
last_verified	Timestamp of last successful binding verification
binding_version	Version of the binding algorithm to support future migration

10.3 KDF Parameters

Field	Description
algorithm	'argon2id'
memory_kib	Argon2id memory parameter in KiB (minimum 65536 recommended)
iterations	Argon2id time cost parameter (minimum 3 recommended)
parallelism	Argon2id parallelism parameter
hash_length	Output length in bytes (32 for AES-256 MUK)
salt_ref	Path to salt.bin on USB device
version	Argon2 version integer (19 = 0x13)

10.4 Object Registry Entry

Field	Description
object_id	UUID v4. Immutable after creation.

object_name	User-visible name (encrypted in full manifest, stored as name_enc)
object_type	MIME type or application type string (e.g. 'application/pdf', 'text/plain', 'obscura/credential')
created_at	Creation timestamp (UTC)
modified_at	Last modification timestamp
size_bytes	Size of plaintext payload in bytes
ciphertext_size	Size of encrypted payload in bytes (may include padding)
payload_ref	Relative path to payload.enc within the object container
meta_ref	Relative path to meta.enc
filekey_ref	Relative path to filekey.enc (or inline if below threshold)
chunk_count	Number of payload chunks if chunked; 0 if single-file
chunk_refs	Array of relative paths to chunk files
tags_enc	Encrypted tag array for search/categorisation
access_policy_id	Reference to object-level policy in policy_metadata
integrity_hash	SHA-256 of ciphertext bytes (also in integrity_hashes map)
is_deleted	Boolean soft-delete flag. True if object pending secure erase.

10.5 Wrapped Key Record

Field	Description
object_id	References the object in the object registry
wrapped_filekey	Base64-encoded AES-256-KW wrapped FileKey bytes
wrap_algorithm	'AES-256-KW' or 'AES-256-GCM' (key wrap algorithm)
key_version	FileKey version counter. Incremented on rotation.
rotated_at	Timestamp of last FileKey rotation (null if never rotated)
filekey_id	Unique ID for this wrapped key record — used in audit trail

10.6 Policy Metadata

Field	Description
vault_export_policy	Global export policy: 'deny_all', 'prompt_each', 'allow_with_audit'
session_timeout_seconds	Idle session timeout. 0 = no timeout (not recommended).
inactivity_lock_seconds	Lock vault after N seconds of inactivity (separate from full session close)
usb_removal_action	Action on USB removal: 'seal_immediately' (required), 'prompt' (not recommended)
recovery_scope_policy	What objects are accessible during a recovery session: 'none', 'credentials_only', 'all'
require_reauth_on_export	Whether re-authentication is required before each export operation
max_recovery_attempts	Maximum failed recovery code attempts before recovery is locked
debugger_response	Response to debugger detection: 'seal' (required in production), 'warn', 'allow' (dev only)
vm_response	Response to VM detection: 'seal', 'warn', 'allow'
object_policies	Map of object_id -> per-object policy overrides

10.7 Signing Reference

Field	Description
signing_key_id	Identifier of the Ed25519 signing key used for this manifest version
signing_cert_ref	Path to or fingerprint of the signing certificate (under root CA)
signature_algorithm	'Ed25519'
signed_at	Timestamp of signing operation

10.8 Rotation Metadata

Field	Description
last_rekey_at	Timestamp of last full vault re-key (MUK change)
rekey_count	Number of times the vault has been re-keyed
filekey_rotation_log	Array of {object_id, rotated_at, key_version} for FileKey rotations
signing_key_rotations	Array of {old_key_id, new_key_id, rotated_at} for signing key rotations

10.9 Recovery References

Field	Description
recovery_bundle_ref	Path to bundle.enc in recovery container
code_verifier_ref	Path to codes.hash in recovery container
recovery_enabled	Boolean — whether recovery access is enabled for this vault
recovery_codes_count	Number of recovery codes issued (hashes stored; codes are out-of-band)
recovery_scope	Object scope during recovery session — as per recovery_scope_policy

10.10 Audit References

Field	Description
audit_log_ref	Path to audit.log.enc in recovery/trace container
last_audit_hash	SHA-256 of the last committed audit log entry (chain anchor)
audit_entry_count	Total number of audit entries — used to detect log truncation
session_log_ref	Path to session.log.enc in trace container

11 Object Model

11.1 Per-Object Encryption

Every vault object is encrypted with a unique FileKey. No two objects share a FileKey. The FileKey is a 256-bit cryptographically random value generated at object creation time. It never appears in plaintext outside application memory during an active session. The FileKey is the only key material needed to decrypt the object's payload — but it is wrapped with the MUK and stored in the manifest, which is itself encrypted with the MUK. This two-layer protection means that even if an attacker somehow recovers the encrypted manifest and the object ciphertext, they still need the MUK to unwrap the FileKey, and they need the USB device to derive the MUK.

11.2 Object Identity and Addressing

Each object has a UUID v4 as its identifier. The UUID is used to: (1) locate the object's directory in the object container; (2) reference the object in the manifest's object registry and wrapped key records; (3) key the integrity hash map; (4) generate the HKDF derivation context for the FileKey when HKDF-based key derivation is used in place of AES-256-KW.

11.3 Small Objects vs. Chunked Objects

Objects below a configurable size threshold (default: 16 MB) are stored as a single payload.enc file. Objects above this threshold are split into fixed-size chunks (default: 4 MB per chunk), each independently encrypted with the same FileKey but with a unique nonce. The chunk manifest is stored in the object's meta.enc and referenced in the manifest's object registry entry.

Each chunk is addressed as chunk_NNNN.enc where NNNN is the zero-padded chunk index. Chunk integrity is verified per-chunk by the AEAD tag. The full object integrity hash in the manifest covers the concatenation of all chunk hashes.

11.4 Object Metadata

Object metadata is stored separately from the payload in meta.enc, encrypted with the FileKey. Metadata includes: object name, MIME type, tags, creation/modification timestamps, plaintext size, ciphertext size, chunk count, and any application-specific metadata. Separating metadata from payload allows metadata lookups without decrypting the full payload — only the metadata FileKey unwrap and metadata AES decryption are required.

11.5 Object Lifecycle State Machine

State	Description

Created	Object exists in manifest; FileKey generated and wrapped; payload written to USB
Closed	Object payload is encrypted on USB; no plaintext in memory; FileKey is wrapped in manifest only
Open	Object has been accessed; FileKey unwrapped into SecureMemory; payload decrypted into SecureMemory
Modified	Object payload has been changed in memory; re-encryption pending
Pending Commit	Modified object being re-encrypted and written to USB; manifest update in progress
Soft-Deleted	Object marked is_deleted in manifest; payload not yet securely erased
Purged	Payload securely erased from USB; manifest entry removed; FileKey zeroed

11.6 Wrapped Key Mapping

The mapping between an object and its FileKey is maintained in the manifest's wrapped_key_records array. Each record contains the object_id, the AES-256-KW wrapped FileKey bytes, the wrap algorithm, and the key version. On object access, Phantom Wrap locates the wrapped key record by object_id, unwraps the FileKey using the in-memory MUK, and returns the FileKey to Phantom Object. On object close, the FileKey is zeroed by Phantom Purge.

12 Key Hierarchy

12.1 Seven KDF Inputs

The Master Unlock Key is derived from seven inputs. All seven must be present and correct for the derivation to succeed. The loss of any input (other than the vault salt, which is non-secret) renders the vault permanently inaccessible.

Input	Description
1. User Passphrase	The user's memorised secret. Collected at unlock; used immediately as KDF input; never stored. Strength is user-controlled but minimum complexity can be enforced by policy.
2. USB Secret	A 32-byte cryptographically random value written to the USB Root Container at vault creation. Never leaves the USB device. Not accessible without physical device access.
3. USB Hardware Fingerprint	A deterministic hash of hardware attributes of the specific USB device (e.g., serial number, vendor ID, product ID, and additional attributes). Captures the physical identity of the device.
4. Hidden Keyfile	A 64-byte entropy amplifier stored in the USB Root Container. Provides additional entropy beyond the passphrase. Acts as a second secret factor alongside the USB binding material.
5. Vault KDF Salt	A 32-byte random salt written at vault creation and stored in salt.bin in the Root Container. Non-secret but unique per vault; prevents rainbow table attacks and ensures different vaults derive different MUKs even with the same passphrase.
6. Argon2id Work Parameters	The memory, iterations, and parallelism parameters configured at vault creation. These are not secret but are part of the KDF input; changing them changes the MUK.
7. Context Binding String	An application-defined domain separation string (e.g. 'phantom-obscura-v2-muk') passed as the Argon2id context or as an HKDF info parameter. Prevents cross-application key reuse.

12.2 MUK Derivation Formula

```
input_material = passphrase || USB_secret || fingerprint_hash || keyfile_bytes  MUK = Argon2id(   password   = input_material,   salt       = vault_kdf_salt,   memory     = kdf_params.memory_kib,   iterations = kdf_params.iterations,   parallelism = kdf_params.parallelism,   length     = 32 )
```

The passphrase, USB_secret, fingerprint_hash, and keyfile_bytes are concatenated in a defined order with length-prefix framing before being passed to Argon2id as the password parameter. This ensures that a change in any one input changes the entire MUK output.

12.3 Per-Object FileKey Derivation

Two approaches are supported for per-object FileKey management:

Option A — Random FileKey with AES-256-KW wrapping: a 256-bit random FileKey is generated per object, wrapped with the MUK using AES-256 Key Wrap (RFC 3394), and the wrapped key is stored in the manifest. On access, AES-256-KW unwrap using the MUK recovers the FileKey.

Option B — HKDF-derived FileKey: the FileKey is derived deterministically using HKDF-SHA256 with the MUK as the input key material and the object UUID as the info parameter. This eliminates the need to store wrapped keys in the manifest for objects but prevents per-object key rotation without re-encrypting the object.

The target architecture uses Option A (random FileKey + AES-256-KW wrapping) as the default because it supports per-object key rotation and recovery scope limiting (specific FileKeys can be excluded from recovery bundles). HKDF derivation may be used for ephemeral objects where rotation is irrelevant.

```
FileKey = Random(32 bytes)                    // Option A wrapped_FileKey =
AES_256_KW(MUK, FileKey)     // stored in manifest  -- OR --  FileKey = HKDF-SHA256(
// Option B   ikm  = MUK,   salt = vault_id,   info = object_id_bytes,   len  = 32 )
```

12.4 Key Material Lifecycle

Key Material	Created
User Passphrase	Entered by user
Master Unlock Key	Argon2id output
Manifest Decryption Output	Manifest load
Ed25519 Signing Key (plaintext)	MUK decrypt of root.key.enc
Per-Object FileKey (unwrapped)	AES-256-KW unwrap
Recovery Session Key	Recovery bundle decrypt

13 Cryptographic Model

13.1 Algorithm Suite

Algorithm	Role	Parameters
Argon2id	Master Unlock Key derivation from passphrase + USB material	memory >= 65536 KiB, iterations >= 3, parallelism >= 1, output 32 bytes
AES-256-GCM	Primary AEAD for all encrypted file storage (objects, manifest, metadata)	256-bit key, 96-bit random nonce, 128-bit authentication tag
ChaCha20-Poly1305	Alternative AEAD (policy-selectable)	256-bit key, 96-bit nonce, 128-bit tag; preferred on platforms without AES-NI
AES-256-KW (RFC 3394)	FileKey wrapping using MUK	256-bit KEK (MUK), 256-bit FileKey; produces 320-bit wrapped output
HKDF-SHA256	Per-context key derivation (FileKey Option B, subkey derivation)	IKM = MUK; salt = vault_id or random; info = object_id or context string
Ed25519	Manifest signing, policy signing, identity signing	255-bit key; 512-bit signature; deterministic; no random nonce required
ECDSA/P-256 + X.509	Certificate chain verification (root CA model)	Used for signing key certificate verification; root cert stored on USB
SHA-256 / BLAKE3	Integrity hashing, fingerprint hashing, audit chain hashing	SHA-256 for integrity hashes in manifest; BLAKE3 for Merkle tree computation
PBKDF2-SHA256	Recovery code verifier hashing	Stored as verifier only; recovery codes are out-of-band
ML-KEM (Kyber)	Post-quantum key encapsulation (future/extension hooks)	Hooks present in service layer; not integrated into primary key hierarchy

13.2 AEAD Nonce Management

All AES-256-GCM and ChaCha20-Poly1305 operations use a 96-bit (12-byte) random nonce generated freshly for each encryption operation. The nonce is prepended to the ciphertext in the stored file. The AEAD authentication tag is appended to the ciphertext. A stored encrypted file therefore has the layout:

```
| nonce (12 bytes) | ciphertext (N bytes) | auth_tag (16 bytes) |
```

Nonce reuse under the same key is catastrophic for AEAD security (it leaks the XOR of plaintexts and invalidates the authentication tag). Phantom Obscura mitigates nonce reuse risk by: (a) using random nonces (not counters) for each encryption; (b) per-object FileKeys ensuring that even if nonces collide across objects, the keys are different; (c) FileKey rotation invalidating all prior nonces for the rotated key.

13.3 Integrity and Merkle Model

Object integrity is maintained at three levels: (1) per-object AEAD authentication tag, verified on every read; (2) per-object integrity hash in the manifest (SHA-256 of ciphertext bytes), checked on object open; (3) Merkle root over all integrity hashes, checked on manifest load to detect tampering with individual object entries.

The Merkle tree is computed over the sorted array of (object_id, ciphertext_hash) pairs. The Merkle root is stored in the manifest and covered by the manifest's Ed25519 signature. Any modification to any object's ciphertext — whether by an attacker or a storage error — produces a ciphertext hash mismatch, which produces a Merkle root mismatch, which causes the session to abort before any data is presented to the user.

13.4 Signing Model

The vault manifest and policy metadata are signed with an Ed25519 key whose certificate is rooted to the USB-resident root certificate. The signing flow is: (1) serialise and encrypt the manifest or policy document; (2) compute Ed25519 signature over the encrypted bytes; (3) store signature as a detached .sig file alongside the encrypted document. On verification: (1) load the encrypted document and its signature; (2) verify the signature against the signing key's certificate; (3) verify the signing certificate against the root certificate; (4) decrypt the document only if all verifications pass.

13.5 Post-Quantum Readiness

The Kit.zip codebase contains references to ML-KEM (Module Lattice Key Encapsulation Mechanism, formerly Kyber) in the service layer and README. The current architecture does not integrate ML-KEM into the primary key hierarchy, but the hooks exist. The intended integration is: the FileKey wrapping operation (currently AES-256-KW) would be augmented with an ML-KEM encapsulation step, producing a hybrid classical/post-quantum wrapped key. This is a planned extension and is not in the current engineering roadmap as a priority milestone.

14 Signing and Trust Model

14.1 Certificate Root

The trust chain is rooted at an X.509 root certificate stored on the USB device at /usb/.phantom/root/root.cert. This root certificate is self-signed and generated at vault creation time. It is specific to the vault and is not a public certificate authority. Its private key (root.key.enc) is stored encrypted on the USB device and is decrypted using the MUK only when a signing operation is required.

The root certificate anchors all signing key certificates. An Ed25519 signing key certificate is issued by the root CA and used for operational signing (manifest signing, policy signing). This separation allows the operational signing key to be rotated without changing the root of trust.

14.2 What Signs What

Document	Signed By	Verification Path
Vault Manifest (encrypted bytes)	Ed25519 operational signing key	Sig verified against signing cert -> root cert -> root CA
Policy Metadata (encrypted bytes)	Ed25519 operational signing key	Same as manifest
Manifest Version Chain Entry	Ed25519 operational signing key	Covers prior manifest hash + new manifest hash; prevents rollback
Audit Log Entry Hash	Ed25519 operational signing key	Covers prior hash + entry; chain integrity verification
Recovery Bundle	Ed25519 operational signing key	Verified before recovery session is permitted
Build Artefacts	Build-time key (different from vault key)	Verified by BuildIntegrityVerifier.cs at application startup
Signing Key Certificate	Root CA private key	Verified against root.cert on USB device

14.3 Canonical Document Definition

The canonical form of the vault manifest is: the encrypted bytes as written to manifest.enc on the USB Vault Container, at the point when the accompanying manifest.sig was produced. Any in-memory representation is derived from this canonical form. Any divergence between the in-memory manifest and the canonical form (as could result from a memory corruption or an in-flight write failure) is detected by re-verification on next load.

14.4 Key Rotation

Operational signing key rotation: a new Ed25519 key pair is generated; a certificate for the new key is signed by the root CA key; all documents signed with the old key are re-signed with the new key; the rotation is recorded in rotation_metadata in the manifest; the old signing key certificate is added to a revocation list in the manifest.

Root CA key rotation: a new root CA key pair is generated; a new root.cert is written to the USB Root Container; all certificates in the chain are re-issued under the new root; this is treated as a vault re-key operation and requires the user's passphrase.

FileKey rotation: a new random FileKey is generated for the object; the object payload is re-encrypted with the new FileKey; the new FileKey is wrapped and the manifest is updated; the old wrapped key record is removed. FileKey rotation can be performed per-object without affecting other objects.

15 Unlock Flow

The unlock flow comprises thirteen steps from application boot to active session. Every step is a gate: failure at any step aborts the flow, logs the event, and returns the application to the unauthenticated state. The flow cannot be partially completed and left in an intermediate state.

Step	Action	Failure Response
1 App Boot	Application starts. BuildIntegrityVerifier.cs verifies the build hash. RuntimeDefence initialised. No vault-specific operations begin until build integrity confirmed.	Abort boot. User notified. No vault access possible.
2 USB Enumeration	Phantom Bind enumerates attached USB devices. Identifies candidate Phantom USB devices by presence of .phantom directory.	No USB found: wait for insertion. Wrong device: prompt user.
3 Fingerprint Pre-check	Phantom Root reads fingerprint.bin from the USB Root Container. Compares USB hardware attributes against stored fingerprint hash.	Fingerprint mismatch: abort; log tamper event; do not proceed.
4 Root Container Open	Phantom Root opens and verifies root container integrity (AEAD tag check). Reads identity.enc, root.cert.	AEAD failure: abort; log integrity error.
5 Identity Verification	Phantom Root decrypts identity.enc using a bootstrap key derived from fingerprint only. Verifies vault_id and binding profile consistency.	Identity mismatch: abort; log event.
6 Passphrase Collection	Application UI collects passphrase from user. Passphrase is held in SecureMemory for the duration of the KDF call only.	User cancel: return to unauthenticated state cleanly.
7 KDF Input Assembly	Phantom Seal reads USB_secret, keyfile, and salt from USB Root Container. Assembles all seven KDF inputs.	Read error: abort; log device error.
8 MUK Derivation	Phantom Seal calls Argon2id with assembled inputs. MUK allocated in SecureMemory. Passphrase and USB_secret zeroed immediately after KDF.	KDF error: abort; zero all inputs; return to unauthenticated state.
9 Manifest Decryption	Phantom Vault reads manifest.enc. Decrypts using MUK (AES-256-GCM). Verifies AEAD tag.	AEAD failure: MUK likely wrong (wrong passphrase); abort; zero MUK; prompt retry.
10 Manifest Signature Verification	Phantom Vault reads manifest.sig. Verifies Ed25519 signature against signing certificate. Verifies certificate against root.cert.	Signature failure: abort; zero MUK; log tamper event; do not load object registry.

11 Merkle Root Verification	Phantom Vault computes Merkle root over all integrity hashes in the loaded manifest. Compares to stored merkle_root.	Merkle mismatch: abort; zero MUK; log integrity violation.
12 Session Initialisation	Phantom Session creates session context. Object registry loaded into session state. USB presence monitor started. Session timers initialised.	Init error: abort; trigger full teardown.
13 Active Session	Vault is fully unlocked. Objects accessible on demand. USB presence monitor running. Runtime Defence monitoring active.	USB removed: immediate seal (Step teardown). Idle timeout: seal. Tamper event: seal and log.

16 Runtime Session Architecture

16.1 Ephemeral Session Model

A Phantom Obscura session is entirely ephemeral. Nothing about the session — no key material, no decrypted object content, no manifest plaintext — persists beyond the session boundary. A session begins at the completion of Step 12 of the unlock flow and ends when any of the session close conditions are triggered.

The session is defined by the in-memory MUK, the in-memory manifest, and the session buffer registry maintained by Phantom Session. These three artefacts together constitute the 'session state'. All three are zeroed on session close.

16.2 Object Access Model (Open on Demand)

Objects are not decrypted into memory at session open. They are decrypted on demand, when the user explicitly opens an object. This open-on-demand model limits the amount of plaintext in memory at any one time. An object that has not been opened in the current session has its FileKey wrapped in the manifest and its payload encrypted on USB — it contributes zero plaintext to the in-memory attack surface.

When an object is opened: Phantom Wrap unwraps the FileKey from the manifest using the MUK. Phantom Object decrypts the payload into a SecureMemory buffer. The buffer is registered with Phantom Session's buffer registry. The FileKey is zeroed after decryption. The decrypted buffer is used by the application.

16.3 Object Close Behaviour

When an object is closed (user action, idle timeout, or session close): if the object has been modified, Phantom Object re-encrypts the buffer with a new FileKey (Option A) or the same HKDF-derived FileKey (Option B), writes the ciphertext to USB, and updates the manifest. Phantom Purge then zeros the plaintext buffer. The wrapped FileKey is updated in the manifest if a new FileKey was generated. The manifest is re-signed and written to USB.

16.4 Session Close Conditions

Close Condition	Trigger
User locks vault	User action in UI
USB device removed	USB removal event from Phantom Bind
Idle timeout	Session timer service (SessionTimerService.cs)
Inactivity lock	Inactivity timer (IdleLockService.cs)

Tamper detection trigger	IntrusionService.cs / TamperDetectionService.cs
Debugger detected	AdvancedDebuggerDetection.cs
Application close	OS window close / process termination

16.5 Re-Authentication vs. Re-Binding

Two modes of session resumption exist after a lock:

Re-authentication: the vault re-derives the MUK from user passphrase + USB material. Required after: idle timeout, user lock, application restart. The USB device remains physically bound; only the MUK needs to be re-derived.

Re-binding: the full unlock flow from Step 2 is executed. Required after: USB device removal and re-insertion; USB device replacement; fingerprint mismatch on re-insertion. Re-binding re-verifies the hardware fingerprint and re-establishes the complete trust chain.

17 Sealed Workspace Model

17.1 In-Memory vs. Controlled Temporary Storage

The default and required behaviour is that all decrypted object content is held in application memory (SecureMemory allocations) only. There is no controlled temporary storage on the host filesystem. Controlled temporary storage — a restricted, application-managed temporary directory — may be used in a future extension for object types that require a subprocess to render them (e.g., a PDF viewer), but this is a non-trivial security extension that requires explicit design and is not in the current scope.

17.2 Open-in-Place Rules

'Open in place' refers to the practice of writing a decrypted copy of a vault object to the host filesystem, opening it in an external application, waiting for changes, and then re-importing and re-encrypting. This pattern is prohibited in the current architecture. All editing of vault objects must occur within the Phantom Obscura application or within an application integration that receives the plaintext data via a defined in-process interface — never through a host-side file drop.

17.3 Forbidden Host-Side Behaviours

Forbidden Behaviour	Risk
Writing decrypted object to temp directory	Temp files survive process termination; indexed by OS
Autosave of in-progress edits to host filesystem	Autosave files persist after crash; not zeroed on session close
Caching decrypted thumbnails or previews	Preview cache survives session; accessible without unlock
Passing plaintext to external processes via filesystem	Child process may not clean up its copies
Logging decrypted object names or content to application logs	Logs may be read by OS tools; not protected by vault encryption
Writing passphrase or key material to any persistent store	Obvious catastrophic key exposure
Clipboard write without guard	Clipboard content accessible by all applications

17.4 Application Integration Policy

Third-party application integration — allowing external applications to receive plaintext from the vault — is architecturally permitted only through a defined IPC interface that: (1) requires the calling application to be registered in the vault policy; (2) uses an encrypted IPC channel; (3) transfers plaintext in a manner that allows Phantom Obscura to audit and control the transfer; (4) tracks the receiving process in SpawnedProcessTracker.cs; (5) terminates the receiving process when the vault seals.

No such IPC interface is currently implemented. Any application integration that does not meet these requirements falls outside the zero-knowledge guarantee.

18 Plaintext Containment

Plaintext containment is the set of active mitigations against OS-level side channels through which decrypted vault content could escape the application boundary. Each leak path listed below is a real and exploited vector in practice. 'Relied on user not saving' is not a mitigation.

Leak Path	Risk Description	Required Treatment	Implementation
Temporary Files	OS and application frameworks write temp files routinely. Decrypted content written to %TEMP% survives process termination and is not zeroed.	No plaintext write to any temp directory. Intercept all OS temp-write APIs.	No plaintext write path in Phantom Store. Framework temp-write interception.
Autosave	Office-style applications autosave documents on timers. If a vault object is opened for editing, autosave would write plaintext to the profile directory.	No autosave-to-host path. All edit state held in SecureMemory.	Application-level autosave disabled for vault objects. No external editor integration currently.
Search Indexing	Windows Search and macOS Spotlight index file contents. If any plaintext reaches the filesystem, it will be indexed within seconds.	File system paths used by the application must be excluded from search index. No plaintext files on host.	Registry exclusion for application directory. No plaintext on host filesystem.
Clipboard History	Windows 11 clipboard history captures clipboard content and syncs to cloud (clipboard history enabled by default). Decrypted content copied to clipboard would be captured.	ClipboardHistoryExclusion.cs registers clipboard operations as excluded from history. ClipboardGuard.cs controls all clipboard writes.	ClipboardGuard.cs (present). ClipboardHistoryExclusion.cs (present, Windows 11 verification needed).
Thumbnail Generation	Windows Explorer generates thumbnail previews of images and documents. If a vault object is written to disk, a thumbnail is generated and	No plaintext object files on host. Thumbnail generation is blocked at source.	No plaintext on host filesystem. Thumbnail generation hook not currently needed given no-plaintext-on-disk guarantee.

	cached.		
Crash Dumps	Windows Error Reporting generates crash dump files containing full process memory on application crash. If the MUK or plaintext objects are in memory during a crash, they appear in the dump.	Register process with WER to suppress or redirect crash dumps. Mark sensitive memory regions to be excluded from dumps.	Not currently implemented. This is a gap. MemoryProtectionService.cs should add WER exclusion hooks.
Pagefile / Swap Residue	Virtual memory management can page out any memory region to the pagefile/swap. SecureMemory.cs must use VirtualLock (Windows) or mlock (Linux) to prevent page-out of sensitive buffers.	All buffers containing MUK, plaintext objects, or FileKeys must be allocated in page-locked memory via SecureMemory.cs.	SecureMemory.cs present. Page-lock coverage of all sensitive allocations needs audit.
Recent Files Lists	Windows Recent Items and application-level recent file lists record file paths and sometimes content previews. If a vault object path is recorded, it leaks the object's existence.	Suppress recent file registration for all vault object accesses.	Application-level suppression needed. Not confirmed in current codebase.
Print Spooler	Printing a vault object sends plaintext to the Windows print spooler, which spools to disk.	Print operations from vault context must encrypt the spool data or prevent spooling.	Print integration not currently implemented. ExportGuard.cs should cover print export path.

19 Runtime Defence Layer

The Runtime Defence layer is a collection of nine active threat-detection and response subsystems operating continuously during a live vault session. Each subsystem is independent. A failure or bypass of one subsystem does not weaken the others. All subsystems report through IntrusionService.cs, which aggregates threat events and determines the appropriate response action.

19.1 Defence Subsystem Summary

Subsystem	Source Files	Threat Addressed	Response on Trigger
Debugger Detection	AdvancedDebuggerDetection.cs	Attached debugger, API monitors, analysis tools inspecting application memory	Immediate vault seal; threat event logged; optionally terminate process
VM Detection	VirtualMachineDetection.cs	Application running inside a virtual machine (analysis environment, sandbox)	Configurable: seal (production), warn (dev); VM-in-VM detection included
Tamper Detection	TamperDetectionService.cs	Application binary modified post-build; DLL injection; hook installation	Seal vault; log tamper event with anomaly details; optionally crash with zero of memory
Build Integrity	BuildIntegrityVerifier.cs	Modified build artefacts shipped or injected; supply chain attack	Refuse to start if build hash fails; no partial start possible
Clipboard Guard	ClipboardGuard.cs	Clipboard sniffers, malicious applications reading clipboard after vault copies	Controls what the application writes to clipboard; clears clipboard on session close
Clipboard History Exclusion	ClipboardHistoryExclusion.cs	Windows 11 clipboard history capturing vault content and syncing to cloud	Registers application clipboard operations as excluded from Windows clipboard history API
Export Guard	ExportGuard.cs	Unauthorised export of vault objects to host filesystem; shadow copy	Enforces vault export policy; blocks export operations that violate policy; logs all export attempts

Intrusion Service	IntrusionService.cs, ThreatEvent.cs	Aggregated threat response — multiple low-severity events may constitute an attack	Threat event aggregation; risk scoring; configurable response actions per threat level
Memory Protection	MemoryProtectionService.cs, SecureMemory.cs	Memory scraping, process memory dump, cold boot attack	Page-locked SecureMemory allocations; process memory access restrictions; MiniDump exclusion
Window Protection	WindowProtectionService.cs	Screen capture, accessibility API abuse reading vault UI content	Marks vault windows as protected from screen capture and accessibility enumeration
Process Tracking	SpawnedProcessTracker.cs	Child processes receiving plaintext and not being cleaned up on vault seal	Tracks all processes spawned by application; terminates children on session close or USB removal

19.2 Threat Event Pipeline

When any defence subsystem detects a threat condition, it creates a ThreatEvent record (ThreatEvent.cs) and submits it to IntrusionService.cs. The IntrusionService evaluates the event against the configured threat policy and determines the response action. Response actions in ascending severity are: log-only, warn-user, lock-session (MUK cleared, re-auth required), seal-vault (full session teardown), emergency-wipe (seal + zero all sensitive memory immediately).

The threat event pipeline ensures that the vault always seals before the user is notified. Notifying first would give an attacker a window to extract memory before the seal completes.

19.3 Build Integrity Model

BuildIntegrityVerifier.cs verifies a cryptographic hash of the application's own binary and its critical DLL dependencies at startup, before any vault operations begin. The expected hash is embedded in a signed manifest distributed with the build. If the computed hash does not match the expected hash, the application refuses to start. This prevents modified builds from accessing vault data.

The current gap: the sign-time hash generation is not integrated into the build pipeline. The hash must be computed, embedded, and signed as part of the build process — not at runtime. This is enumerated as a gap in Section 25.

19.4 Memory Protection Details

SecureMemory.cs provides the page-locked memory allocator used for all sensitive in-memory data: the MUK, the manifest plaintext, decrypted object buffers, and FileKeys. Page-locked memory prevents the OS from swapping the content to the pagefile under memory pressure. On allocation, the memory is zeroed. On deallocation, it is zeroed again before the pages are unlocked.

The current gap: the coverage of page-locked allocation is not comprehensively audited. Components that allocate plaintext buffers using standard heap allocations (e.g. through .NET's built-in string or byte array types) are outside the protection boundary. A full audit is needed to ensure all sensitive allocations use SecureMemory.

20 Recovery Architecture

20.1 Recovery Plane Overview

The Recovery Plane provides a constrained, audited pathway back into the vault when the standard unlock path is unavailable. The most common scenarios are: (1) the user has forgotten their passphrase; (2) the USB device has partial corruption affecting only the secret material (USB_secret or keyfile) but the object container is intact. The Recovery Plane is not a backdoor — it requires both: (a) physical possession of the USB device; and (b) a valid recovery code.

20.2 Recovery Code Architecture

Recovery codes are generated at vault creation time by RecoveryCodeService.cs. The application generates N codes (default: 8). The raw codes are presented to the user once and never stored by the application. The user is responsible for storing them securely out of band (printed, offline storage). The application stores only the PBKDF2-SHA256 hash of each code in codes.hash on the USB Recovery Container.

Recovery codes are single-use. When a recovery code is used, its verifier hash is marked as consumed in the recovery container and in the manifest. A consumed code cannot be reused. If all codes are consumed, recovery is disabled until the user generates a new set (which requires a successful standard unlock session).

20.3 Recovery Session Flow

1. User selects recovery mode in the application UI.
2. Phantom Bind verifies USB device presence and fingerprint (hardware binding still required).
3. User enters a recovery code.
4. RecoveryCodeService.cs hashes the entered code and compares against the verifier table. If no match: increment failure counter; abort after max_recovery_attempts.
5. On match: the recovery bundle (bundle.enc) is decrypted using a key derived from the recovery code + USB fingerprint.
6. The recovery bundle contains a constrained recovery key: a version of the MUK encrypted with recovery-specific key material, scoped to the object access policy defined at vault creation.
7. RecoverySession.cs creates a recovery session with the constrained key and restricted object registry (as per recovery_scope_policy).
8. All access during the recovery session is logged to audit.log.enc by RecoveryAuditService.cs.
9. At recovery session end: the used recovery code verifier is marked consumed. A new standard unlock is required to access full vault functionality.

20.4 What Recovery Cannot Bypass

Bypass Attempted	Why It Cannot Be Bypassed
USB hardware binding	Recovery still requires physical USB device presence. The recovery bundle is decrypted using key material that includes the USB fingerprint. No USB = no recovery.
Object-level access policy	Recovery scope is defined in the policy metadata and enforced by the constrained recovery key. Objects excluded from recovery scope cannot be accessed during a recovery session regardless of the recovery code.
Audit trail	All recovery session events are appended to the encrypted audit log. The audit log chain is anchored in the manifest. It cannot be cleared without a full vault re-key and manifest re-signing under the standard unlock path.
Manifest integrity	The recovery key decrypts only the constrained recovery bundle, not the full manifest. Object integrity verification (Merkle root check) still runs for accessible objects.
Single-use code enforcement	Code verifier is marked consumed in the recovery container atomically on first use. A race condition that reads the code before it is marked consumed would require simultaneous multi-process access to the USB device.

20.5 Separation from Standard Unlock

The recovery path is entirely separate from the standard unlock path at the code level. RecoverySession.cs is not a subclass of the standard session. RecoveryCodeService.cs does not interact with KdfOrchestrator.cs or MasterUnlockKeyService.cs. The recovery key material is distinct from the MUK and cannot be used to re-derive the MUK. A successful recovery session does not expose the MUK and does not allow the user to extract FileKeys for objects outside the recovery scope.

21 Deception Layer

21.1 Overview

The Deception Layer provides a credible false vault experience that can be presented to an attacker who has obtained the user's passphrase through coercion, observation, or social engineering — but who does not know that a real vault exists beneath. The deception layer does not protect against cryptographic attacks; it protects against coercion attacks where the user is compelled to reveal their passphrase.

21.2 Decoy Vault Architecture

DecoyVaultService.cs manages a decoy vault experience. When the decoy trigger condition is met, the application presents a convincing but non-functional vault populated with synthetic content generated by DecoyCredentialGenerator.cs. The decoy vault:

- Looks and behaves identically to a real vault at the UI level.
- Contains synthetic credentials, documents, and objects that are plausible but non-functional.
- Is populated with plausible but fake data including realistic-looking passwords, addresses, notes, and documents.
- Does not reveal the existence of the real vault.
- Records interaction with the decoy vault in an isolated audit log.

21.3 Decoy Trigger Conditions

Trigger	Description
Decoy passphrase	A second, separate passphrase is configured at vault creation. Entering this passphrase opens the decoy vault instead of the real vault. The decoy passphrase is indistinguishable from the real passphrase from the attacker's perspective.
Tamper threshold exceeded	If the intrusion service has flagged multiple anomaly events suggesting coercion (e.g., unusual unlock time, repeated failed attempts followed by success), the application may auto-route to the decoy vault.
Explicit user activation	User can pre-configure the decoy vault to be shown under specific conditions (e.g., after N failed attempts before success).

21.4 Isolation from Real Vault

The decoy vault is cryptographically and physically isolated from the real vault. DecoyVaultService.cs does not hold references to the real vault's MUK, manifest, or object

container paths. The decoy content is generated independently and stored separately. There is no pathway from the decoy session to the real vault's key material.

Interacting with the decoy vault does not trigger any signal that would reveal to the attacker that it is a decoy. The application response time, UI behaviour, and error handling in the decoy session are designed to be indistinguishable from a real session.

21.5 Relation to Tamper and Intrusion Services

The Deception Layer is loosely coupled to the Intrusion Service. ThreatEvent.cs events may inform the routing decision (real vault vs. decoy vault) but do not trigger decoy activation automatically in all cases. The primary activation mechanism is the decoy passphrase. The Intrusion Service's role is to monitor whether a decoy session is being probed (e.g., an attacker testing whether the decoy is real by attempting operations) and to log those events.

22 Threat Model

This section enumerates the threat actors and threat scenarios relevant to Phantom Obscura. For each threat, the attack description, the primary mitigations, and the residual risk are identified.

22.1 Threat Actors

Actor	Description
Opportunistic attacker	Has physical or remote access to a device that has been left unattended. No prior knowledge of the vault or its contents. Looking for easily accessible data.
Targeted attacker (offline)	Has obtained the USB device without the passphrase. May attempt to brute-force, reverse-engineer the container format, or recover data from the USB storage chip.
Targeted attacker (online, no credentials)	Has access to the running host machine (malware, remote access) while the vault is sealed. Cannot decrypt anything without the USB device and passphrase.
Targeted attacker (online, session active)	Has access to the running host machine while a vault session is active. This is the highest-capability attacker; the runtime defence layer is specifically designed for this scenario.
Coercion attacker	Has physical control of the user and demands the passphrase. The deception layer provides partial mitigation through the decoy vault.
Insider / device seizure	A law enforcement or adversarial entity has seized both the host machine and the USB device. Has time and resources to perform forensic analysis.
Supply chain attacker	Has modified the Phantom Obscura application binary before distribution. Build integrity verification and code signing are the primary mitigations.

22.2 Threat Scenarios

Threat	Attack Description	Primary Mitigations
T1 — Offline USB Extraction	Attacker extracts USB device and images all data. Attempts to decrypt containers using brute force or known-plaintext attacks.	AES-256-GCM encryption with Argon2id KDF. MUK cannot be derived without passphrase + USB_secret + fingerprint + keyfile simultaneously. Brute force is computationally infeasible at correct Argon2id parameters. No plaintext or key material on USB in

		accessible form.
T2 — Host Memory Scraping	Attacker has malware on the host and scrapes application memory during an active session looking for the MUK or plaintext objects.	SecureMemory page-locked allocations prevent swap exposure. MemoryProtectionService.cs restricts process memory access. Runtime Defence detects analysis tools. Ephemeral decryption minimises plaintext surface. MUK zeroed on USB removal.
T3 — Passphrase Capture	Attacker captures the passphrase via keylogger, shoulder surfing, or phishing. Attempts to use passphrase without USB device.	Passphrase alone is insufficient to derive MUK. USB device must be present. USB_secret and keyfile are on device. Passphrase capture alone yields zero vault access.
T4 — USB Cloning / Spoofing	Attacker creates a bit-for-bit copy of the USB device and attempts to use the copy to unlock the vault, bypassing the hardware fingerprint requirement.	Hardware fingerprint is computed from hardware-specific attributes of the original USB chip. A clone may pass bit-level checks but fail hardware fingerprint verification if the fingerprint includes attributes derived from the USB controller's internal identity. Fingerprint algorithm strength determines resistance.
T5 — Manifest Tampering	Attacker with USB access modifies manifest.enc or replaces it with a forged manifest to redirect object access or inject malicious objects.	Manifest is protected by both AES-256-GCM AEAD tag (modification detected) and Ed25519 signature (forgery requires signing key). Signing key is encrypted with MUK and stored on USB. Merkle root detects tampering with object integrity hashes.
T6 — Debugger / Analysis Attack	Attacker attaches a debugger or analysis tool to the Phantom Obscura process during an active session to dump memory or modify execution.	AdvancedDebuggerDetection.cs triggers vault seal on debugger detection. API-level monitoring hooks also detected. Response is seal-before-alert to prevent memory access window.
T7 — VM-Based Analysis	Attacker runs Phantom Obscura inside a virtual machine to analyse behaviour, dump hypervisor-accessible memory, or intercept I/O.	VirtualMachineDetection.cs detects VM environment. Configurable response: seal in production (recommended). Hypervisor memory access remains a residual risk if the VM detection is bypassed.
T8 — Recovery Path	Attacker obtains recovery codes (out-	Recovery codes are single-use.

Abuse	of-band) and uses them to access the vault.	Recovery still requires physical USB presence. Recovery scope limits what is accessible. All recovery access is audited. Brute-forcing recovery codes is mitigated by max_recovery_attempts and PBKDF2 verifier hashing.
T9 — Coercion Attack	Attacker has physical control of user and demands vault access. User is compelled to provide passphrase.	Decoy vault activated by separate decoy passphrase. Attacker receives plausible but false content. Real vault is not accessible with decoy passphrase. Residual risk: attacker who knows about the decoy feature may not be satisfied by the decoy.
T10 — Supply Chain Attack	Modified Phantom Obscura binary distributed or injected that exfiltrates vault material.	BuildIntegrityVerifier.cs verifies binary hash at startup. Code signing of release builds. User should verify binary hash out-of-band on first installation. Residual risk: if build integrity check is bypassed or not present.

23 Attack Surface Analysis

The attack surface analysis identifies every interface through which an attacker could interact with or influence the Phantom Obscura system. For each surface, the entry point, attack vector, and mitigation are described.

Surface	Entry Point	Attack Vector	Mitigation
USB Physical Interface	Physical USB connector	Device theft, device cloning, counterfeit device, USB firmware attack	Hardware fingerprint verification; AES-256-GCM encrypted containers; passphrase required; no KDF material derivable from USB alone
Application Memory	Process memory (local or remote)	Memory scraping via malware, debugger attachment, process injection, cold boot	SecureMemory (page-locked); DebuggerDetection; MemoryProtection; ephemeral decryption; zero-on-close
Keyboard / Input Layer	User input events	Keylogger (hardware or software) capturing passphrase	Passphrase alone insufficient (USB required); application may use secure input controls; no password stored anywhere
Host Filesystem	Application working directory, temp dirs	Plaintext temp files, autosave files, OS-generated previews/thumbnails	No plaintext on host filesystem by architecture; no write path to temp dirs
Clipboard	OS clipboard API	Clipboard sniffers, malicious app reading clipboard	ClipboardGuard.cs; ClipboardHistoryExclusion.cs; clipboard cleared on session close
OS APIs (Search, Index, WER)	SearchIndexer, Windows Error Reporting, Recent Files	Search index capturing plaintext; crash dump exposing memory; recent files leaking object existence	Application directory excluded from search; crash dump suppression (gap — not fully implemented); recent file suppression
IPC / Inter-Process	Named pipes, shared memory, COM	Malicious process requesting data from Phantom Obscura via IPC	SpawnedProcessTracker.cs; no unauthenticated IPC endpoints; ExportGuard.cs blocks filesystem-path exports
Network Interface	Any network-capable component	Exfiltration of key material or plaintext via network; command-and-control	No network operations required or performed during vault access; no network listener; firewall policy enforceable at OS level
Recovery Path	Recovery entry	Recovery code brute	Single-use codes; attempt

	point in UI	force; stolen out-of-band recovery codes	limits; USB presence required; audited; constrained scope
Build / Distribution	Application binary	Modified binary with exfiltration payload; DLL hijacking	BuildIntegrityVerifier.cs; code signing; out-of-band hash verification by user
Screen / Window	Screen capture APIs	Accessibility tools, screen capture software reading vault UI	WindowProtectionService.cs marks vault windows as screen-capture protected
Virtual Machine Boundary	Hypervisor memory access	Hypervisor reads VM memory including page-locked pages (some hypervisors can bypass VirtualLock)	VirtualMachineDetection.cs + seal; residual risk: some hypervisors defeat VirtualLock — deep VM detection required

24 Code-Versus-Target Gap Analysis

This section documents the gaps between the current Kit.zip codebase capabilities and the agreed target architecture. Each gap is classified by priority and has a specific resolution action. This analysis is the consolidated output of all ten source documents and the architectural review.

Gap Area	Current Code Status	Target Architecture	What Still Needs Doing
Three-container physical enforcement	Container support exists in ContainerIoService.cs and UsbContainerManager.cs. However, the three-container split (Root / Vault / Object as physically separate containers on USB) is not enforced at the storage layer. The code may read from and write to a single USB path.	Root Container, Vault Container, and Object Container must be physically separate encrypted storage units at distinct paths on the USB device. All three must be present and verified before unlock can proceed.	Implement separate container open/verify/close operations for each of the three containers. Enforce three-container presence check in Phantom Bind before KDF inputs are assembled.
Manifest physical separation	VaultManifestService.cs manages the manifest but may read from and write to a host-side or shared USB path rather than a dedicated Vault Container.	The manifest (manifest.enc, manifest.sig) must live exclusively in the Vault Container at /usb/.phantom/vault/. No copy on host. No caching to host-side storage.	Audit all manifest read/write paths. Ensure all writes go to /usb/.phantom/vault/. Remove any host-side manifest cache or temp file.
No-plaintext persistence guarantee	ObjectEncryptionService.cs and VaultObjectService.cs handle encryption/decryption. However, it is not confirmed that all code paths avoid writing plaintext to the host filesystem. Framework defaults (e.g. .NET file dialog temp files) may leak.	Zero plaintext at rest on host. All object I/O goes to USB encrypted containers. No temp file writes.	Audit all file I/O operations in the codebase. Identify any host-side write path for object content. Add integration test that verifies no plaintext file appears on host after object access.
Strict ephemeral workspace	SessionManager.cs and EphemeralBufferService.cs implement session-scoped buffers. However,	All plaintext buffers must be registered with Phantom Session and	Audit all byte[] and string allocations that may hold decrypted content. Replace with SecureMemory. Add session close

	the buffer registry completeness is not confirmed — not all components may register their plaintext allocations.	zeroed on session close. SecureMemory must be used for all sensitive allocations.	test that verifies all registered buffers are zeroed.
Immediate USB removal reaction	UsbRemovalHandler.cs and SessionInvalidator.cs exist. However, the latency from USB removal event to vault seal completion (all buffers zeroed) has not been measured or guaranteed.	USB removal triggers immediate vault seal. All plaintext zeroed within a bounded time (target: < 100 ms from removal event to first zero operation beginning).	Add instrumented USB removal test. Measure seal latency. Ensure removal handler has highest priority in the event loop. Document guaranteed maximum seal latency.
Single authoritative signing model	Ed25519 signing services exist (RootCertificateService.cs, signing references throughout). However, there are multiple signing models coexisting in the codebase (certificate-root, direct key signing, HMAC integrity) without a clear authoritative model.	One canonical signing model: Ed25519 + X.509 certificate chain rooted at USB-resident root certificate. All manifests, policies, and audit entries signed under this model. HMAC integrity supplementary only.	Choose and document the canonical signing model. Remove or demote competing models. Ensure all manifest writes use the canonical signing flow. Verify certificate chain on every manifest load.
Separate physical storage domains	The three trust domains (Root / Vault / Object) are logically defined but may not be physically enforced. File writes may go to a shared USB path.	Physical path enforcement: Root Container at .phantom/root/, Vault Container at .phantom/vault/, Object Container at .phantom/objects/. Container I/O service enforces path prefix routing.	Implement path-prefix enforcement in ContainerIoService.cs. No writes to a container's path from a service that does not own that container. Unit test each container path in isolation.
Crash dump exclusion	MemoryProtectionService.cs and SecureMemory.cs provide page-locked memory. However, Windows Error Reporting crash dumps can expose page-locked memory in certain configurations.	Application registers with WER to suppress or redirect crash dumps for the process. Sensitive memory regions are annotated for dump exclusion where the API	Add WER crash dump suppression registration in application startup (SetErrorMode / WerRegisterExcludedMemoryBlock). Test with deliberate crash during active session to verify no plaintext in dump.

		supports it.	
Published manifest schema	Manifest structure implied by VaultManifestService.cs and ManifestSerializer.cs. No published, versioned schema with explicit field definitions.	A versioned, published manifest schema (as defined in Section 10 of this specification) that is the canonical definition for all implementation.	Implement manifest schema as a versioned data model (C# record or schema file). Ensure serialisation and deserialisation are schema-driven. Version field in manifest enables migration.

25 Engineering Roadmap

The engineering roadmap defines the eight milestone items required to close the gaps identified in Section 24 and bring the codebase into conformance with the target architecture. Items are ordered by priority: the first three are foundational and must be completed before the others have full meaning.

Milestone	Description	Acceptance Criteria
M1 — Three-Container Physical Enforcement	Implement the three-container USB topology as physically separate encrypted storage units at defined paths. Enforce presence and integrity verification of all three containers before any KDF inputs are read. Implement path-prefix enforcement in ContainerIoService.cs.	(1) Unit tests confirm each container is accessed only through its owning service. (2) Integration test confirms unlock is refused if any container is missing. (3) AEAD tag failure on any container aborts unlock.
M2 — Manifest Physical Separation and Signing Model	Move all manifest read/write operations to the Vault Container. Remove any host-side manifest copy or cache. Implement the canonical Ed25519 + X.509 certificate chain signing model as the sole signing authority. Ensure all manifest writes use sign-then-encrypt and all manifest reads use decrypt-then-verify.	(1) No manifest file appears on host filesystem during or after a session. (2) All manifest writes produce a valid manifest.sig. (3) A tampered manifest.enc causes unlock to abort at signature verification.
M3 — No-Plaintext Persistence Audit	Audit all file I/O operations in the codebase. Identify and eliminate all code paths that write decrypted content to the host filesystem. Add integration tests that monitor host filesystem for plaintext after object access.	(1) Automated test: create vault, open object, close session — assert no new files on host containing object content. (2) No .NET framework default temp writes identified. (3) Clipboard cleared on session close (automated test).
M4 — SecureMemory Coverage Audit	Audit all byte[] and string allocations that may hold passphrase, MUK, FileKeys, or decrypted object content. Replace all such allocations with SecureMemory. Implement buffer registry completeness check.	(1) All KDF inputs use SecureMemory. (2) MUK is in SecureMemory from derivation to zeroing. (3) All FileKeys are in SecureMemory for their lifetime. (4) Session close test verifies all registered buffers are zeroed.
M5 — Crash Dump and OS Side-Channel Suppression	Register with WER to suppress crash dumps for the process. Add WerRegisterExcludedMemoryBlock for	(1) Deliberate crash during active session produces no crash dump

	SecureMemory allocations. Suppress recent files registration. Add instrumented USB removal test with seal latency measurement.	or produces dump without sensitive memory regions. (2) USB removal to first zero operation measured < 100 ms. (3) No recent file entries created during session.
M6 — Versioned Manifest Schema	Implement the full manifest schema defined in Section 10 as a versioned C# record model. Implement schema-driven serialisation and deserialisation. Implement schema migration for older vault formats.	(1) Manifest deserialiser validates against schema on load. (2) Schema version field present and checked. (3) Schema v1 vaults can be migrated to v2 format with explicit user confirmation.
M7 — Recovery Plane Hardening	Implement single-use code enforcement atomically. Implement scope enforcement in RecoverySession.cs that prevents access to objects outside the recovery scope even if the session key is extracted. Wire recovery audit entries into the main audit chain.	(1) Recovery code used once is refused on second attempt. (2) Objects outside recovery scope return access denied during recovery session. (3) All recovery events appear in main audit log.
M8 — Build Integrity Pipeline	Integrate BuildIntegrityVerifier.cs into the build pipeline: compute canonical binary hash at build time, embed in signed build manifest, verify at startup. Implement code signing for release builds.	(1) Modified binary refused at startup. (2) Build pipeline produces signed hash manifest. (3) Release builds have code signing certificate. (4) Out-of-band hash verification instructions published.

26 Public Positioning

This section defines the security claims that Phantom Obscura can make publicly and those that require additional engineering work before they can be made honestly. Distinguishing between defensible claims and aspirational ones is essential for building user trust.

26.1 Claims Supportable Now (Current Codebase)

Claim	Basis
Hardware-bound vault: requires physical USB device to unlock	USB binding and hardware fingerprint verification are present and operational in the codebase.
AES-256-GCM encryption for all stored objects	ObjectEncryptionService.cs uses AES-256-GCM. Confirmed in multiple source documents.
Argon2id key derivation with configurable work factor	KdfOrchestrator.cs and Argon2idService.cs implement Argon2id. Present in codebase.
Per-object key isolation: each object encrypted with a unique key	FileKeyService.cs and KeyWrappingService.cs implement per-object FileKeys.
Ed25519 manifest signing	Signing services present in codebase.
Active runtime defence: debugger detection, VM detection, tamper detection	AdvancedDebuggerDetection.cs, VirtualMachineDetection.cs, TamperDetectionService.cs all present.
Clipboard protection	ClipboardGuard.cs and ClipboardHistoryExclusion.cs present.
Session-scoped decryption: objects decrypted on demand and zeroed after use	SessionManager.cs and EphemeralBufferService.cs implement this model.
Decoy vault for coercion protection	DecoyVaultService.cs and DecoyCredentialGenerator.cs present.
Recovery plane with audited access	RecoverySession.cs, RecoveryAuditService.cs present.

26.2 Claims Requiring Additional Engineering Before They Can Be Made

Claim	What is Required
Zero plaintext on host: the host machine never receives plaintext at rest	Complete M3 (No-Plaintext Persistence Audit). Automated test evidence required before claiming.

Three-container physically isolated USB architecture	Complete M1. Physical enforcement not yet verified.
Signed manifest with certificate-chain verification	Complete M2. Canonical signing model must be implemented and tested.
Page-locked memory for all sensitive data: immune to swap exposure	Complete M4 (SecureMemory Coverage Audit). Coverage gaps exist.
Crash dump suppression: memory not exposed through OS crash handling	Complete M5. WER suppression not yet implemented.
Merkle tree object integrity verification	Merkle integration is incomplete (MerkleService.cs referenced as partial).
Single-use recovery codes with atomic enforcement	Complete M7. Single-use enforcement gap identified.
Build integrity verification integrated into CI pipeline	Complete M8. Currently not wired.
Post-quantum readiness: ML-KEM integration	Post-quantum hooks exist but ML-KEM is not integrated into the primary key hierarchy. Cannot claim PQ-readiness without completing the hybrid encapsulation design.

26.3 Recommended Public Positioning Statement

Phantom Obscura is a hardware-bound encrypted vault designed around a zero-knowledge architecture. Vault contents are encrypted with AES-256-GCM using keys derived through Argon2id from a combination of user passphrase and USB-resident material. No vault key or decrypted content is stored on the host machine. All decryption is ephemeral and session-scoped. The vault includes active runtime defences against debuggers, virtual machine analysis, and clipboard capture, and a deception layer to protect against coercion.

This statement is defensible from the current codebase. The expanded claims around physical three-container isolation, comprehensive no-plaintext guarantees, and crash dump suppression should be added to the public statement only after the corresponding engineering milestones (M1, M2, M3, M5) are completed and validated.

Appendix A — Glossary

Term	Definition
AEAD	Authenticated Encryption with Associated Data. An encryption mode that provides both confidentiality and integrity (authentication tag). AES-256-GCM and ChaCha20-Poly1305 are AEAD ciphers.
Argon2id	A memory-hard password hashing / key derivation function. The 'id' variant combines the side-channel resistance of Argon2i with the GPU resistance of Argon2d. Recommended by OWASP for password hashing and KDFs.
BLAKE3	A fast cryptographic hash function, used for Merkle tree computation in Phantom Obscura.
Control Plane	The trust domain corresponding to the host machine. Treated as hostile.
Ed25519	An elliptic-curve digital signature scheme based on the Edwards 255-19 curve. Used for manifest and policy signing.
FileKey	A 256-bit symmetric key unique to each vault object. Encrypts the object payload. Stored wrapped in the vault manifest.
Hardware Fingerprint	A deterministic hash of USB device hardware attributes used to bind a vault to a specific physical device.
HKDF	HMAC-based Key Derivation Function (RFC 5869). Used for per-context subkey derivation from the MUK.
Hidden Keyfile	A 64-byte random binary blob stored in the USB Root Container. Acts as an additional entropy source in the MUK derivation.
KDF	Key Derivation Function. A function that derives cryptographic key material from a secret (such as a passphrase) and a salt.
Manifest	The encrypted authority structure stored in the USB Vault Container that governs all vault operations.
Merkle Root	A single hash representing the entire tree of object integrity hashes. Used to detect any modification to the manifest's integrity hash table.
ML-KEM	Module Lattice Key Encapsulation Mechanism (formerly Kyber). A post-quantum key encapsulation algorithm standardised by NIST.
MUK	Master Unlock Key. The 256-bit key derived from the seven KDF inputs. Used to decrypt the vault manifest and unwrap per-object FileKeys.
Object Container	The USB container holding all encrypted vault object files.
Payload Plane	The trust domain corresponding to the USB Object Container.

Recovery Plane	The trust domain providing constrained, audited re-entry to the vault.
Root Container	The USB container holding identity material, hardware fingerprint, hidden keyfile, and KDF salt.
SecureMemory	A page-locked memory allocator (SecureMemory.cs) that prevents OS page-out and ensures explicit zeroing on deallocation.
Trust Plane	The trust domain corresponding to the USB Root and Vault Containers.
USB Binding Profile	The record of hardware fingerprint and device identity stored in the vault manifest, used to verify that the correct USB device is present.
Vault Container	The USB container holding the encrypted vault manifest.
Wrapped Key	A FileKey encrypted (wrapped) with the MUK using AES-256-KW. Stored in the manifest; unwrapped during object access.
Zero-Knowledge	As used in this document: the host machine holds no plaintext vault content at rest and derives no persistent cryptographic material. Distinct from the formal cryptographic proof concept of zero-knowledge proofs.

Appendix B — File Reference Index

Key source files from Kit.zip referenced throughout this specification:

File	Layer / Subsystem	Primary Role
AdvancedDebuggerDetection.cs	Runtime Defence	Debugger presence detection and session seal trigger
Argon2idService.cs	Phantom Seal	Argon2id KDF implementation wrapper
AuditLogService.cs	Phantom Trace	Encrypted audit log write operations
AttachmentService.cs	Phantom Object	Object attachment management
BuildIntegrityVerifier.cs	Runtime Defence	Application binary hash verification at startup
ChunkingService.cs	Phantom Object / Store	Large object chunked I/O
ClipboardGuard.cs	Runtime Defence	Clipboard write control
ClipboardHistoryExclusion.cs	Runtime Defence	Windows clipboard history exclusion registration
ContainerIoService.cs	Phantom Store	USB container file I/O
DecoyCredentialGenerator.cs	Deception Layer	Synthetic credential generation for decoy vault
DecoyVaultService.cs	Deception Layer	Decoy vault session management
DevicePresenceMonitor.cs	Phantom Bind	Continuous USB device presence monitoring
EphemeralBufferService.cs	Phantom Session	Session-scoped plaintext buffer registry
ExportGuard.cs	Runtime Defence	Vault export policy enforcement
FileKeyService.cs	Phantom Wrap	Per-object FileKey generation and lifecycle
HardwareFingerprint.cs	Phantom Root	USB hardware fingerprint acquisition
HkdfService.cs	Phantom Seal	HKDF-SHA256 subkey derivation
IdleLockService.cs	Phantom Session	Inactivity-based session lock timer
IntrusionService.cs	Runtime Defence	Threat event aggregation and response
KdfOrchestrator.cs	Phantom Seal	KDF input assembly and MUK derivation

		coordination
KeyRotationService.cs	Phantom Wrap	FileKey rotation operations
KeyWrappingService.cs	Phantom Wrap	AES-256-KW wrap/unwrap operations
ManifestIntegrityService.cs	Phantom Vault	Manifest AEAD and Merkle integrity verification
ManifestSerializer.cs	Phantom Vault	Manifest serialisation and deserialisation
MasterUnlockKeyService.cs	Phantom Seal	MUK lifecycle management
MemoryProtectionService.cs	Runtime Defence	Process memory access restriction
MerkleService.cs	Phantom Vault	Merkle tree computation and verification
ObjectEncryptionService.cs	Phantom Object	Object payload encryption/decryption
ObjectMetadataService.cs	Phantom Object	Encrypted object metadata management
ObjectRegistryService.cs	Phantom Vault	In-memory object registry operations
PasskeyService.cs	Extension Hooks	Passkey / FIDO2 extension hook
PayloadAddressingService.cs	Phantom Object	Object payload path resolution
PolicyEnforcementService.cs	Phantom Vault	Vault policy evaluation and enforcement
RecoveryAuditService.cs	Recovery Plane	Recovery session audit logging
RecoveryCodeService.cs	Recovery Plane	Recovery code generation and verification
RecoverySession.cs	Recovery Plane	Constrained recovery session management
RootCertificateService.cs	Phantom Root	X.509 root certificate management
ScopedRecoveryService.cs	Recovery Plane	Recovery scope enforcement
SecureMemory.cs	Runtime Defence / Phantom Purge	Page-locked memory allocation
SecurePurgeService.cs	Phantom Purge	Multi-pass secure buffer zeroing
SessionAuditService.cs	Phantom Trace	Per-session audit event recording
SessionManager.cs	Phantom Session	Session lifecycle and state management
SessionTimerService.cs	Phantom Session	Session idle timeout management
SpawnedProcessTracker.cs	Runtime Defence	Child process lifecycle tracking and termination

TamperDetectionService.cs	Runtime Defence	Application tamper detection
ThreatEvent.cs	Runtime Defence	Threat event data model
UsbBindingService.cs	Phantom Bind	USB device binding verification
UsbContainerManager.cs	Phantom Store	USB container lifecycle management
UsbIdentityService.cs	Phantom Root	USB device identity record management
UsbRemovalHandler.cs	Phantom Bind	USB device removal event handler
VaultManifestService.cs	Phantom Vault	Vault manifest load, decrypt, verify, write
VaultObjectService.cs	Phantom Object	Vault object lifecycle coordination
VaultSealService.cs	Phantom Seal	Vault seal/unseal lifecycle
VirtualMachineDetection.cs	Runtime Defence	Virtual machine environment detection
WindowProtectionService.cs	Runtime Defence	Window screen-capture protection
YubiKeyService.cs	Extension Hooks	YubiKey hardware token extension hook
ZeroingService.cs	Phantom Purge	Targeted memory zeroing operations